

**bok25**

**基  
础**

**班、计算机组成原理**



# 指令流水线



# 指令流水线

## 流水段的基本构成

流水段的基本构成

取指令(IF):根据PC的值从存储器取出指令

指令译码/读寄存器(ID):产生控制信号, 同时从寄存器中取数据

执行/计算地址(EX):执行运算操作或计算访存地址

访存(MEM):对存储器进行读/写操作

写回(WB):将操作结果写入存储器或寄存器



# 指令流水线

## 流水段的基本构成

取指令(IF)

指令译码(ID)

执行(EX)

访存(MEM)

写回(WB)



# 指令流水线

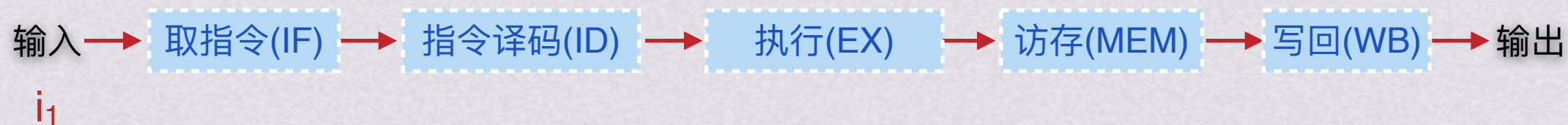
## 流水段的基本构成

取指令(IF)    指令译码(ID)    执行(EX)    访存(MEM)    写回(WB)



# 指令流水线

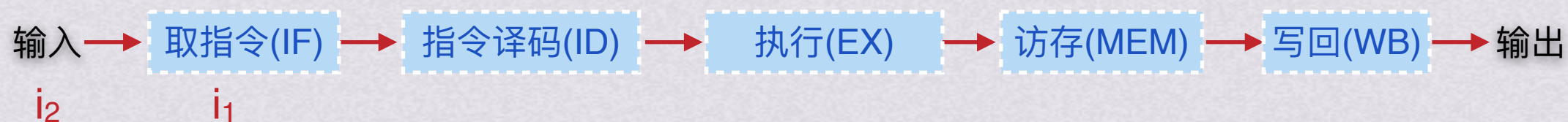
## 流水段的基本构成

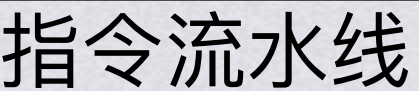




# 指令流水线

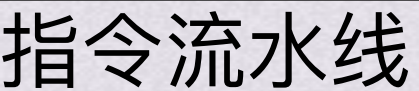
## 流水段的基本构成



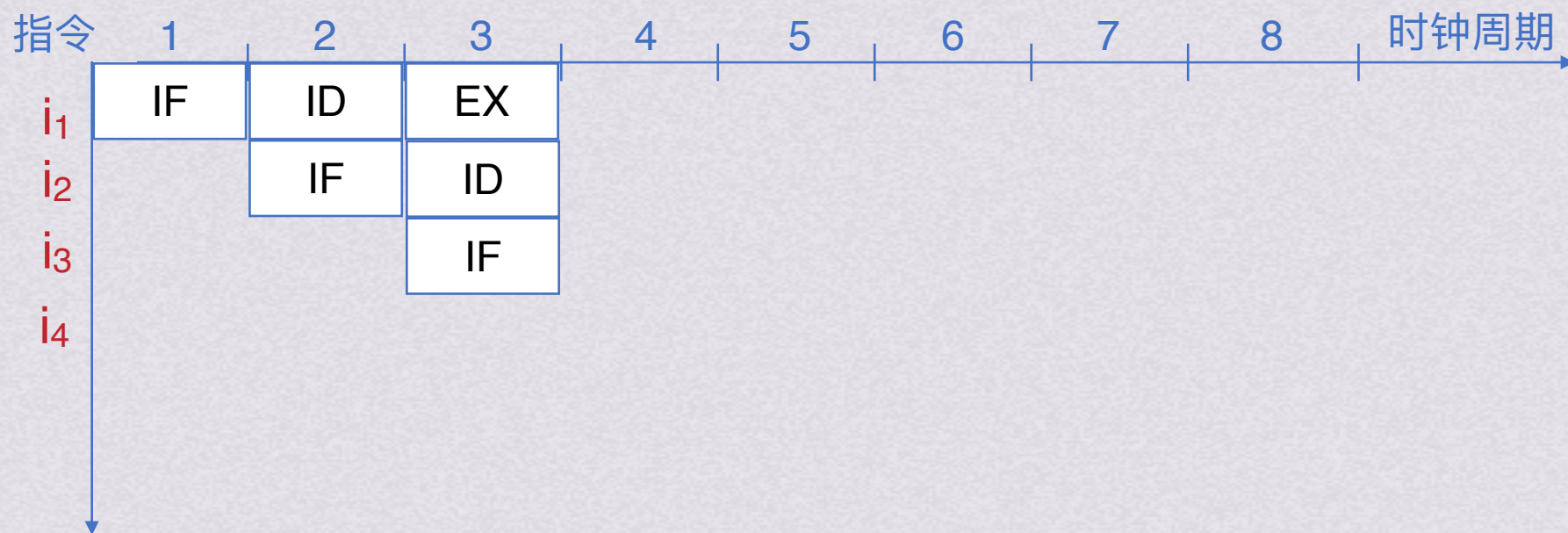


## 流水段的基本构成





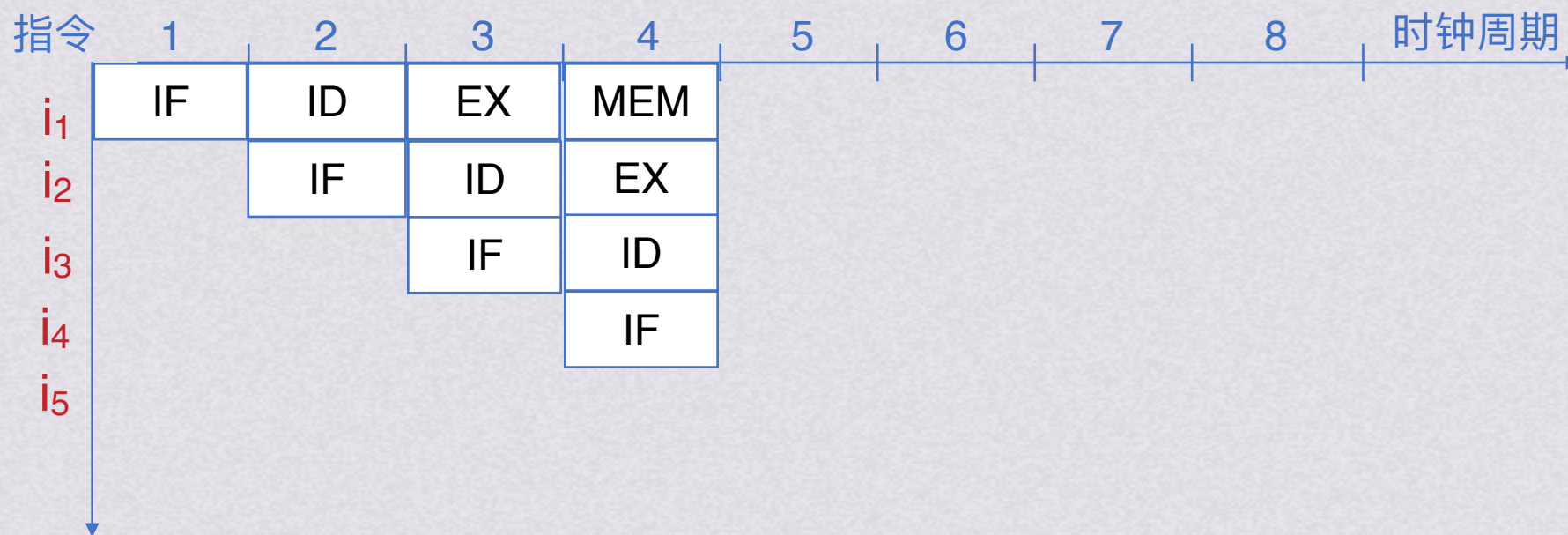
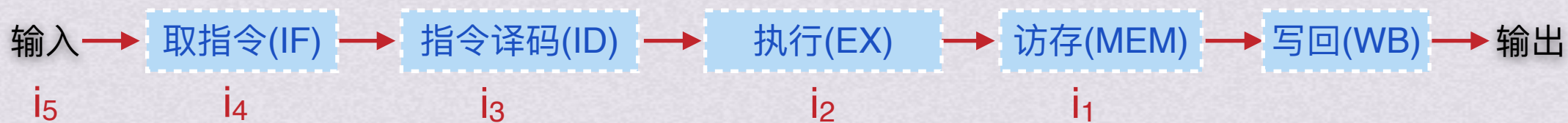
## 流水段的基本构成

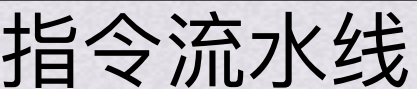




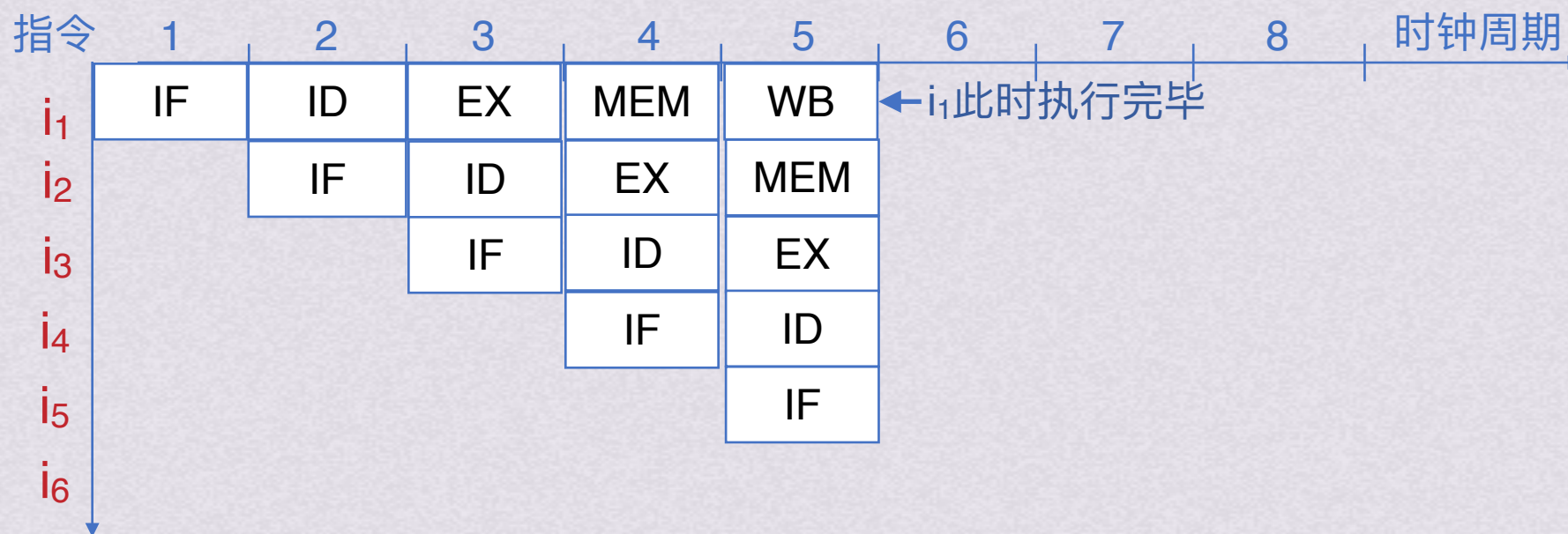
## 指令流水线

## 流水段的基本构成





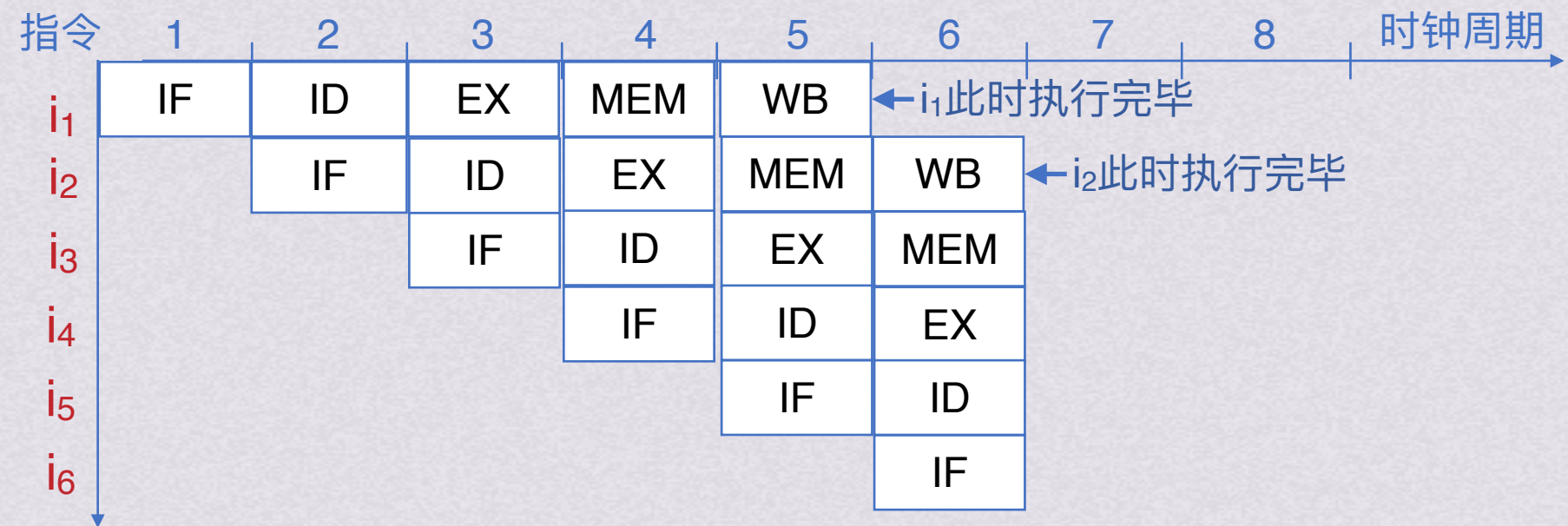
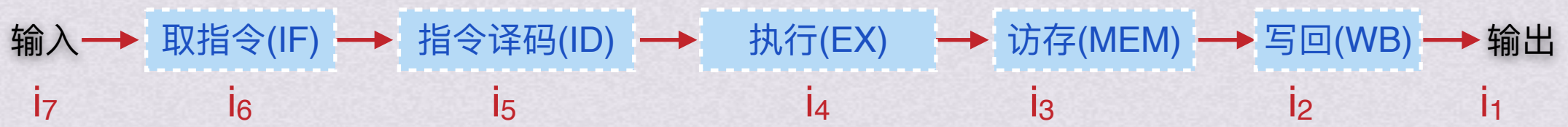
## 流水段的基本构成





# 指令流水线

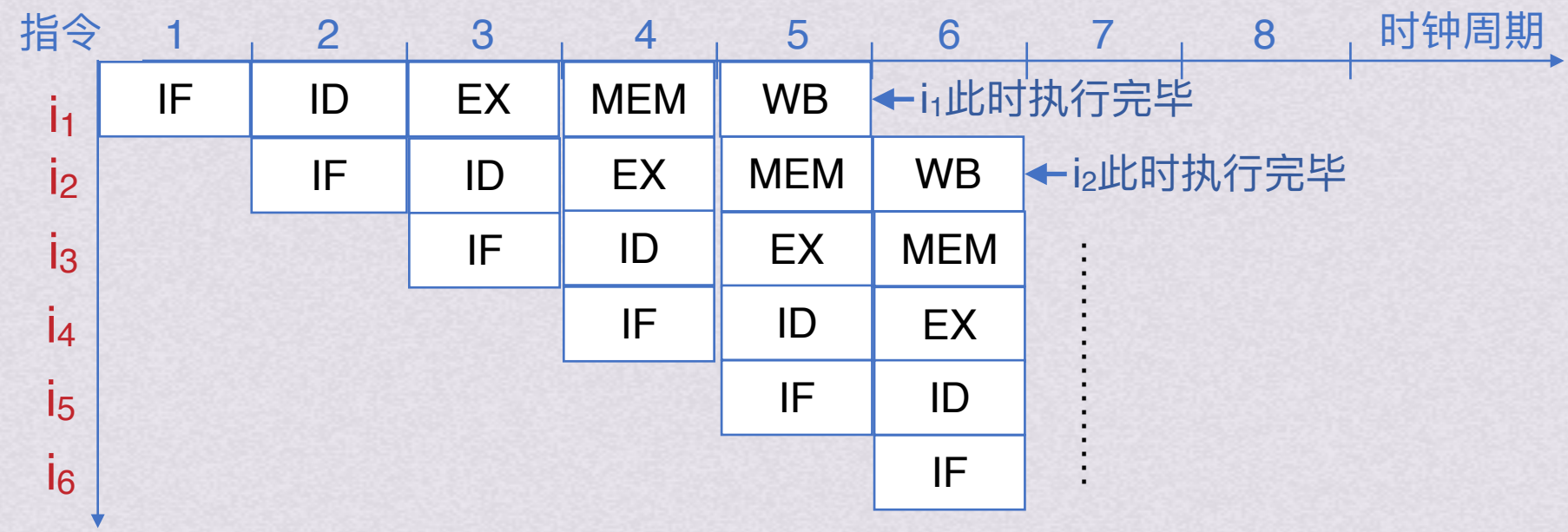
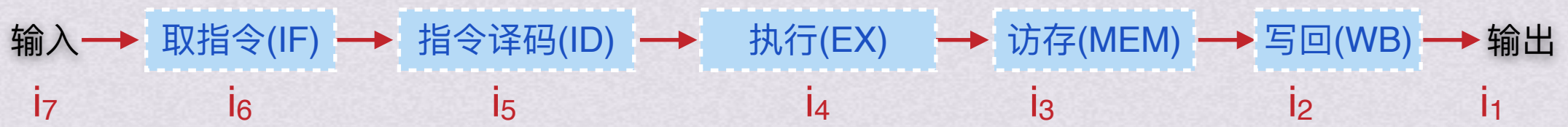
## 流水段的基本构成





# 指令流水线

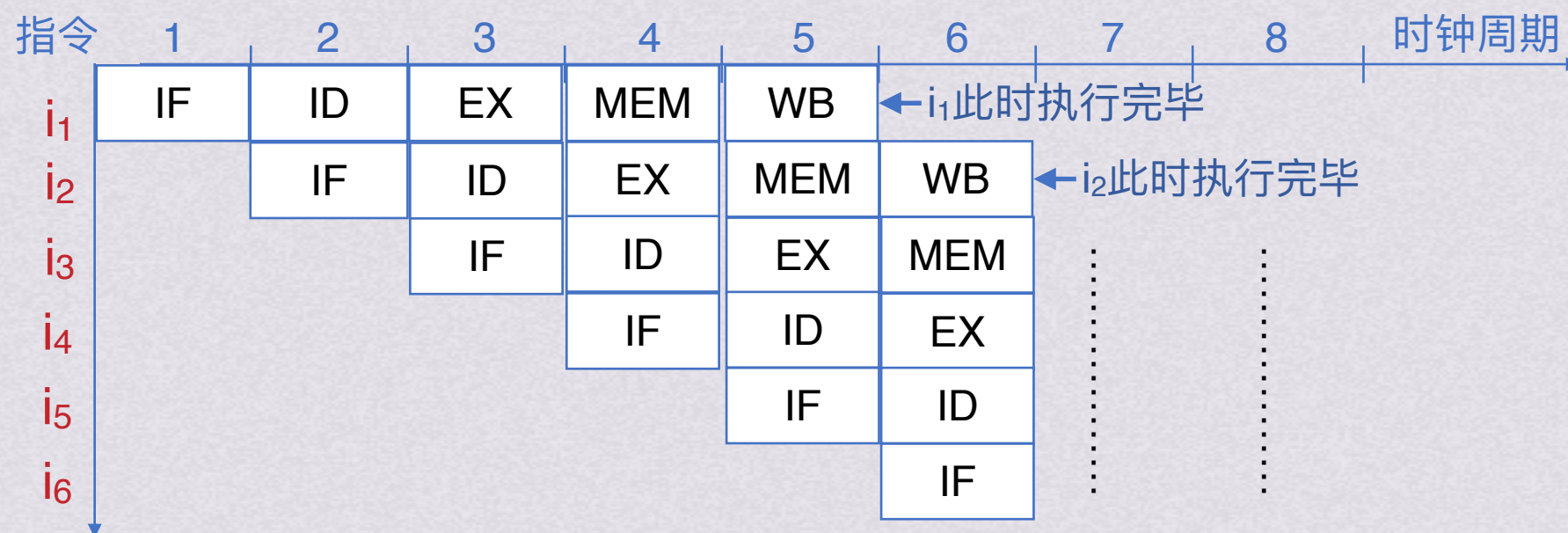
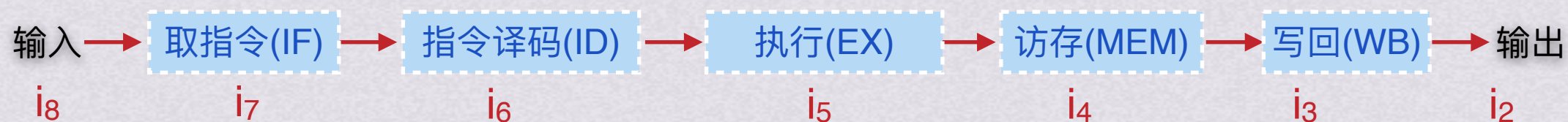
## 流水段的基本构成





## 指令流水线

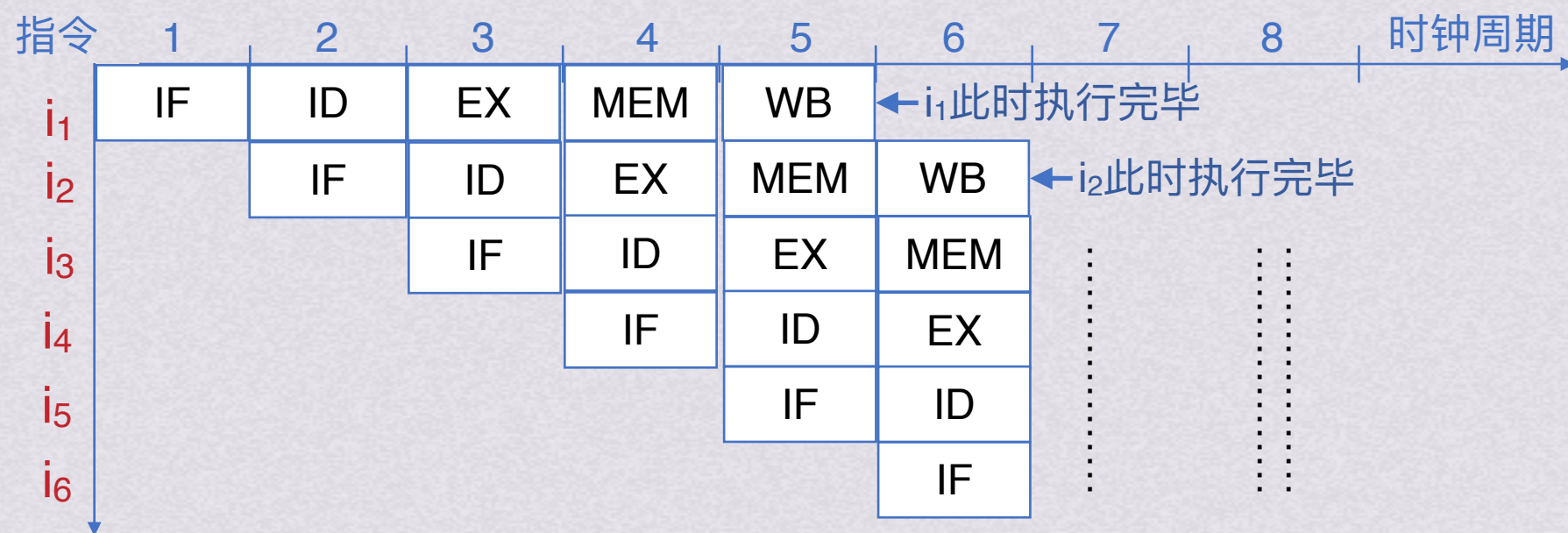
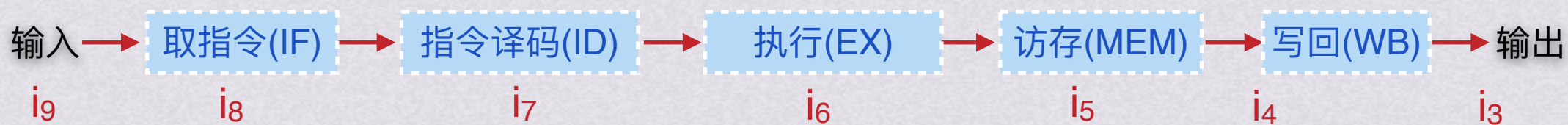
## 流水段的基本构成





## 指令流水线

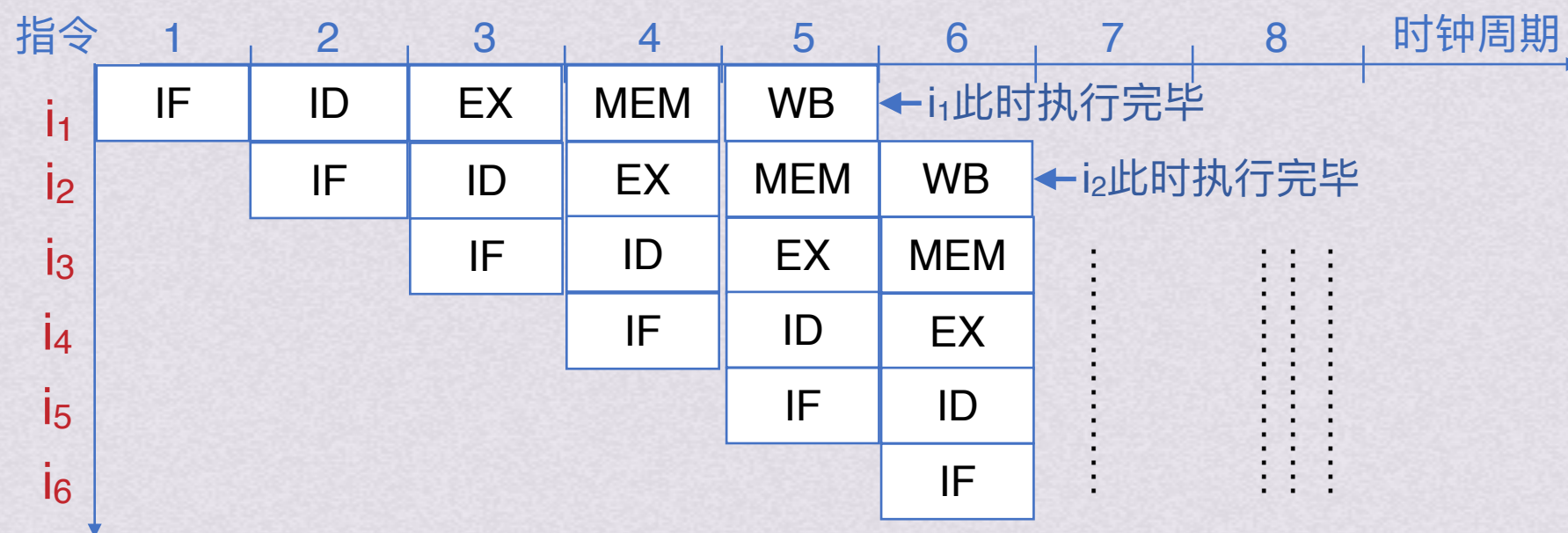
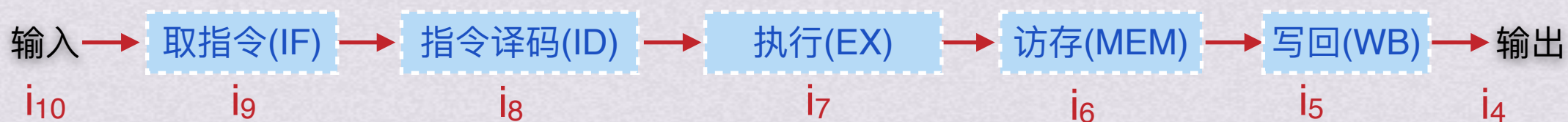
## 流水段的基本构成





## 指令流水线

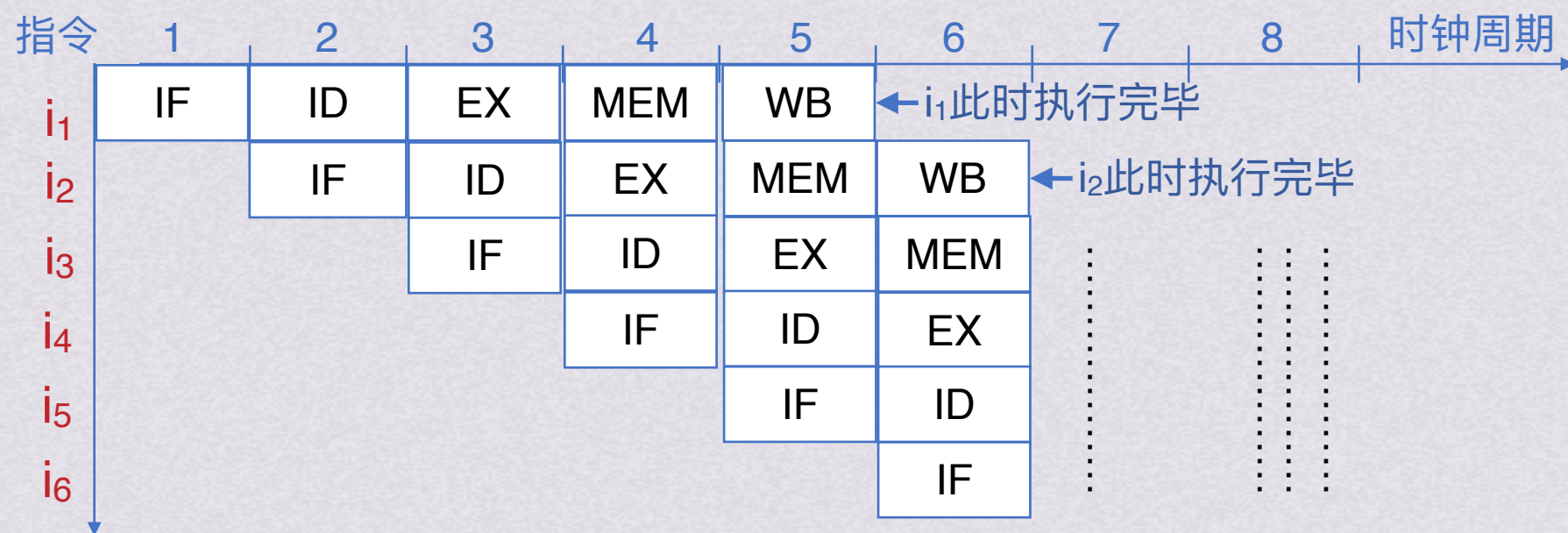
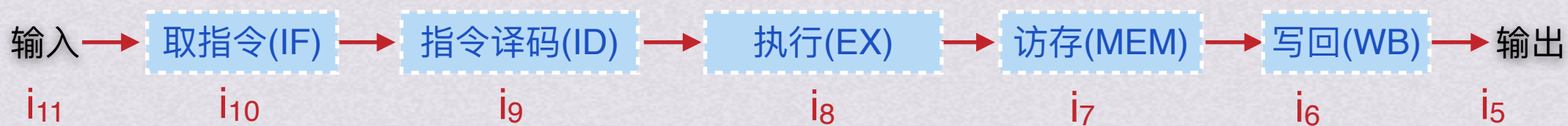
## 流水段的基本构成





## 指令流水线

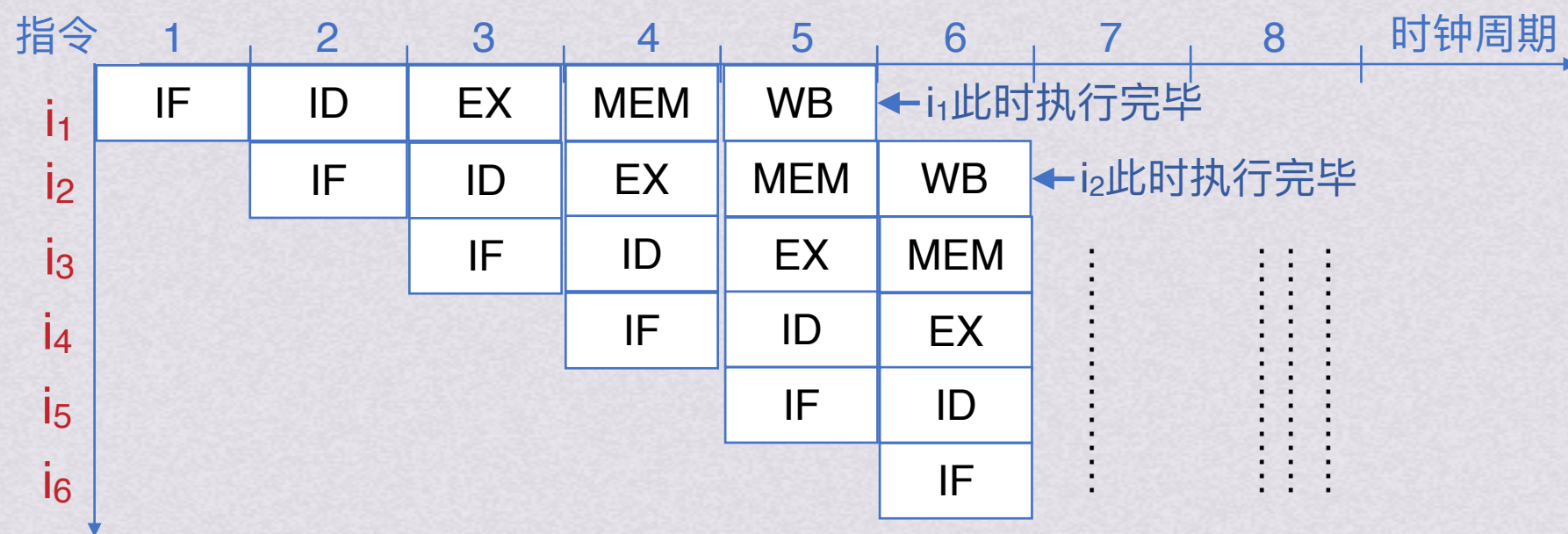
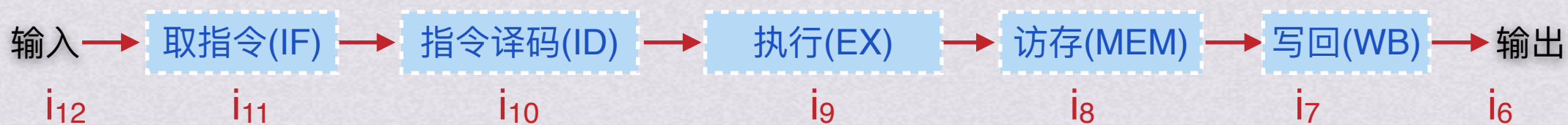
## 流水段的基本构成





## 指令流水线

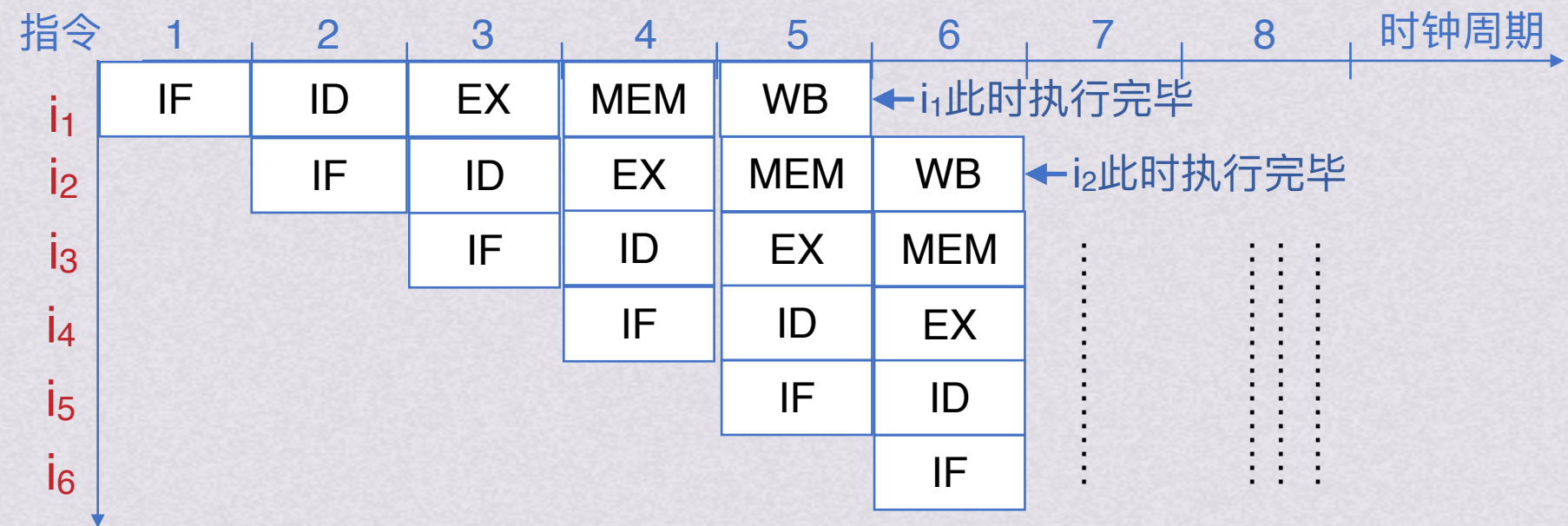
## 流水段的基本构成





# 指令流水线

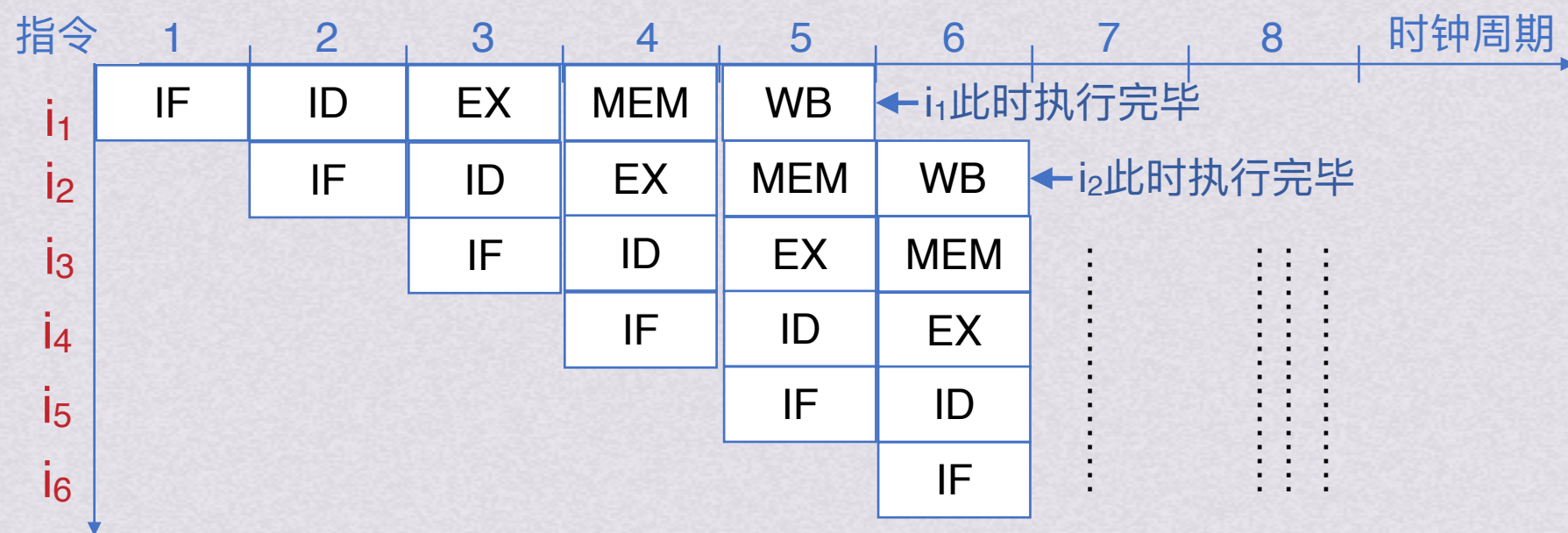
## 流水段的基本构成





## 指令流水线

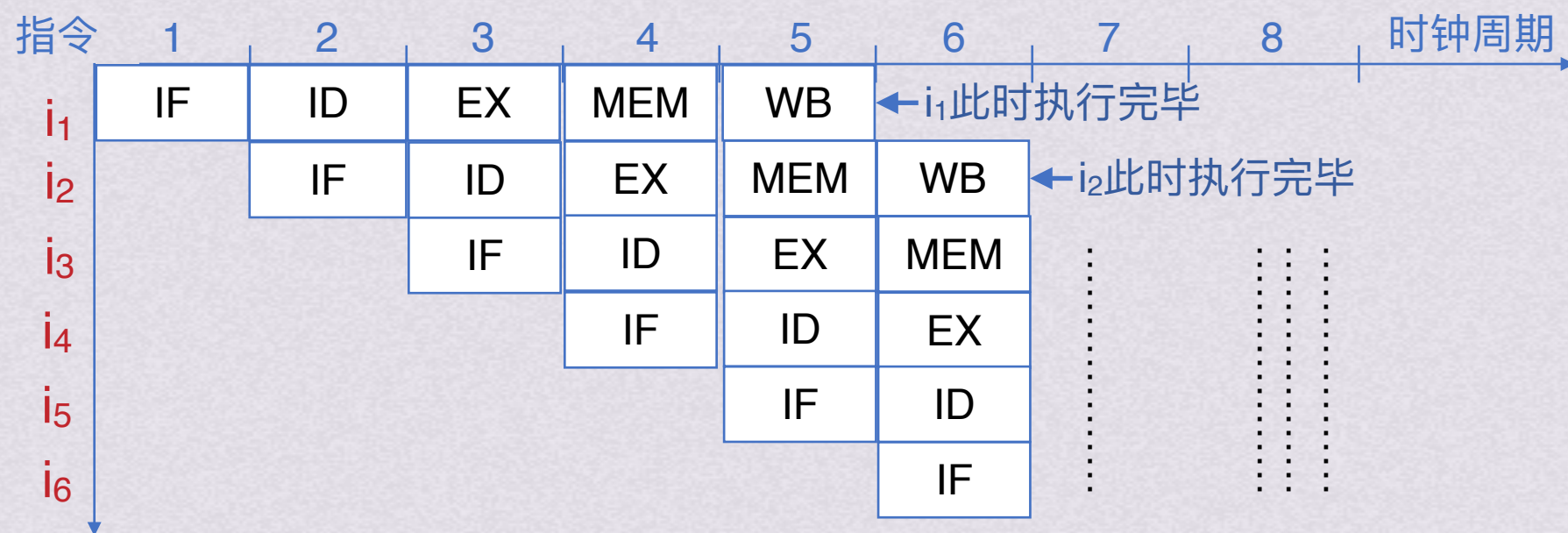
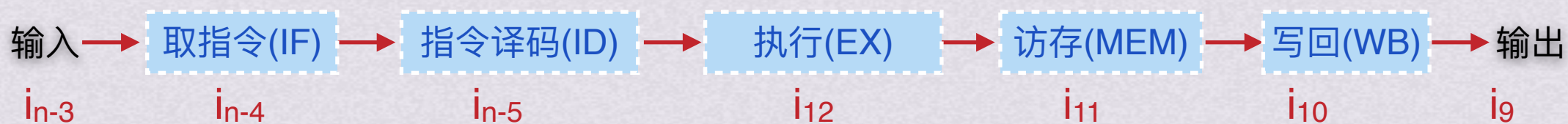
## 流水段的基本构成





## 指令流水线

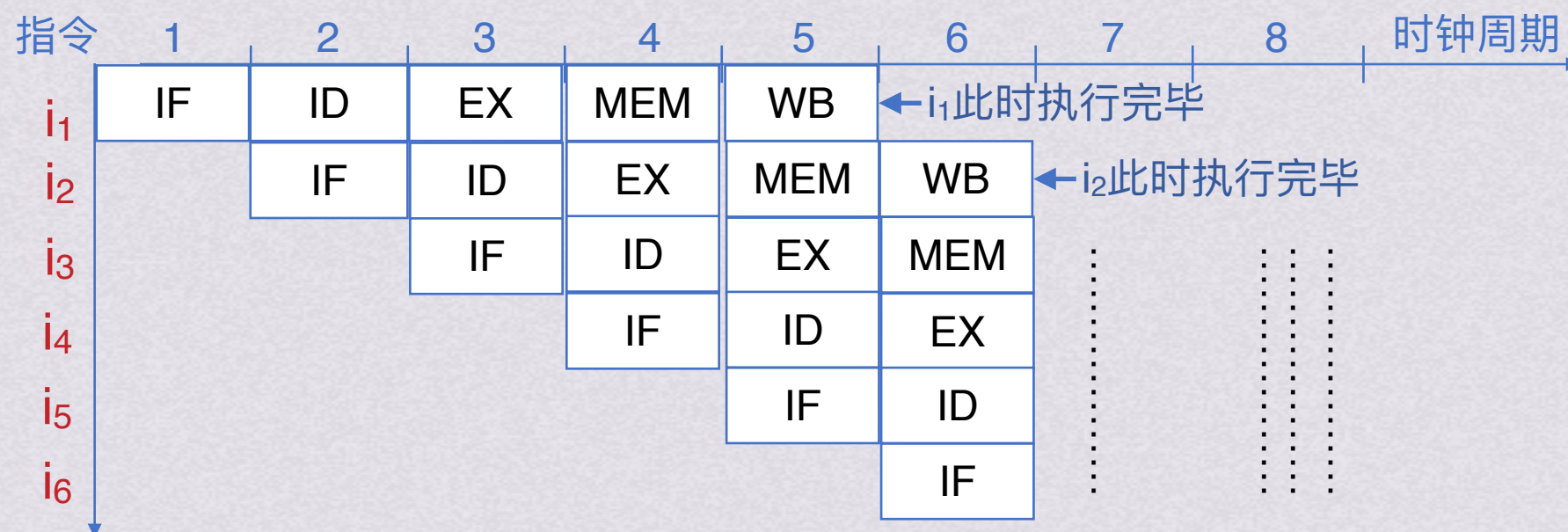
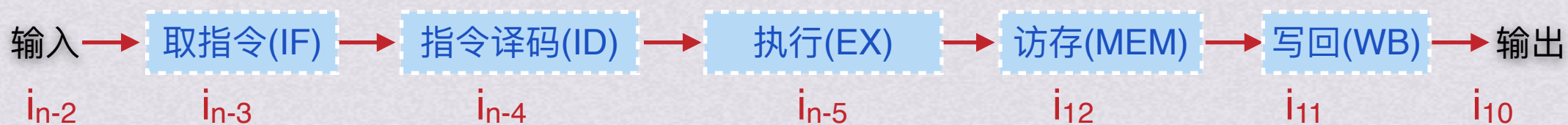
## 流水段的基本构成





## 指令流水线

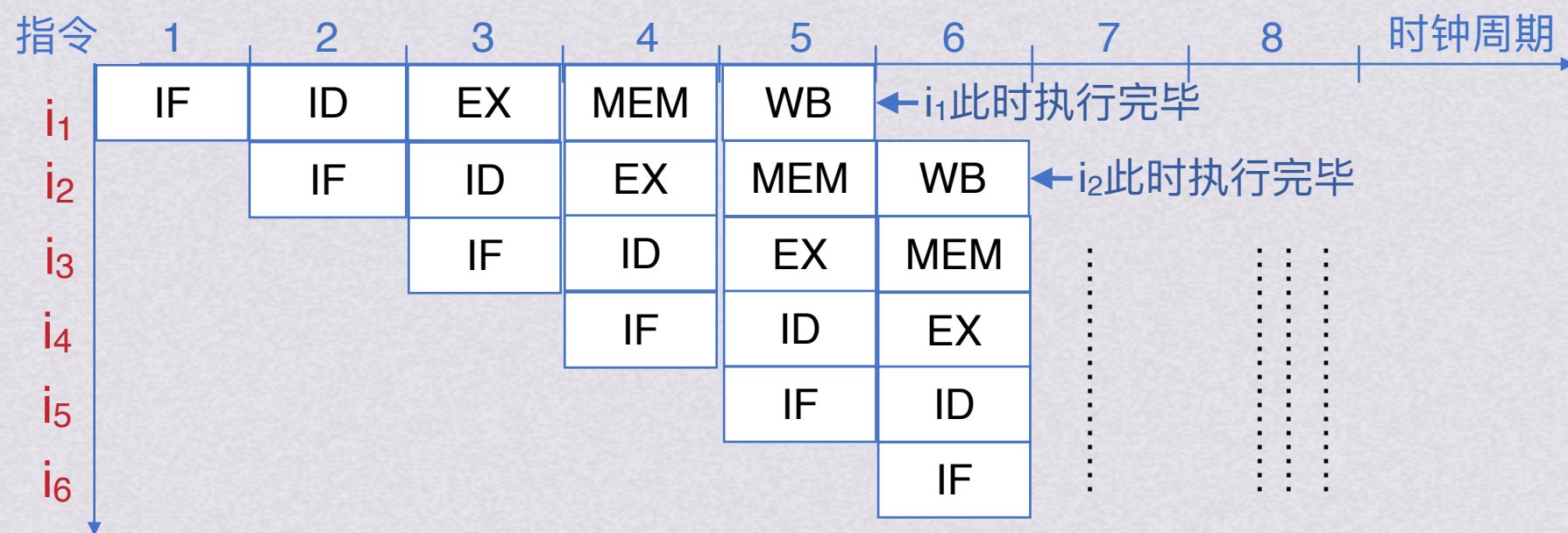
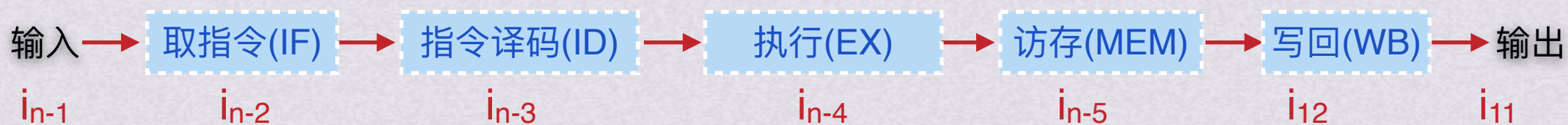
## 流水段的基本构成





## 指令流水线

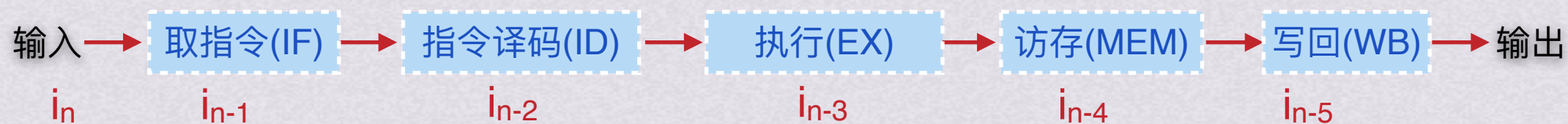
## 流水段的基本构成



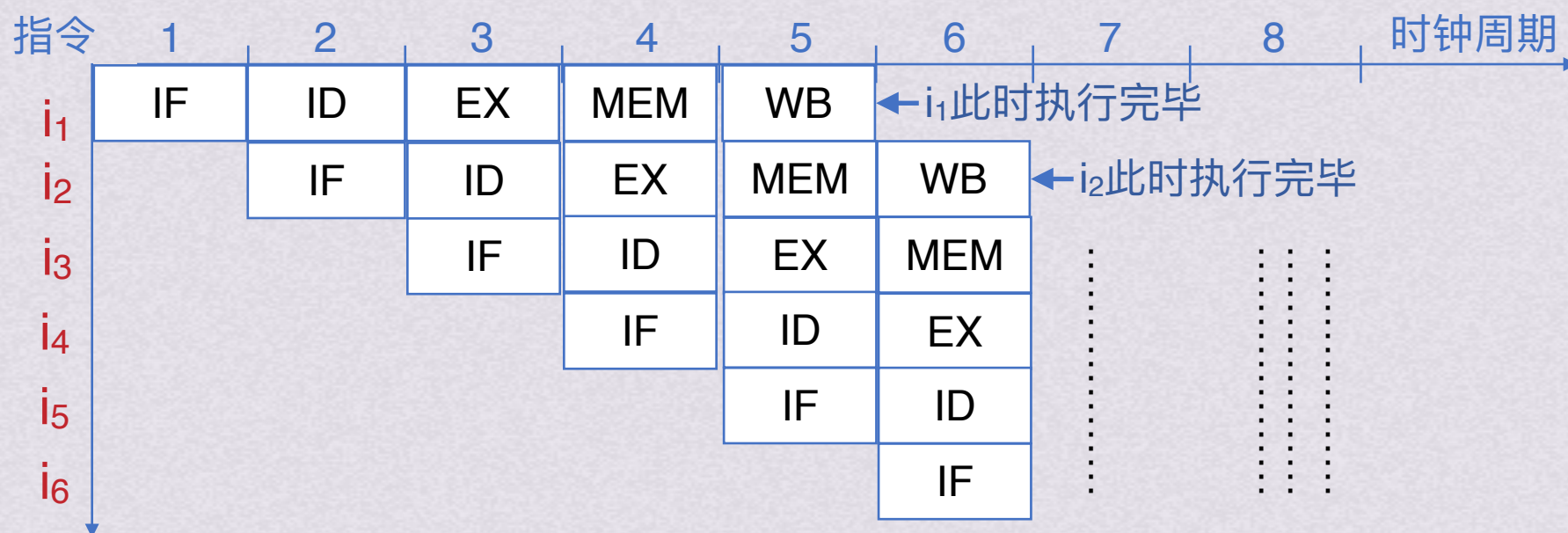


## 指令流水线

## 流水段的基本构成



可以看出，理想状况下，每一个时钟周期都有一条指令进入流水线，每个时钟周期都有一条指令完成





# 指令流水线

## 流水段的基本构成



流水线的数据通路设计原则:

五段流水线

指令流水段个数以最复杂指令所用的功能段个数为准

流水段的长度以最复杂功能段的操作所用时间为准



# 指令流水线

## 流水段的基本构成



流水线的数据通路设计原则:

五段流水线

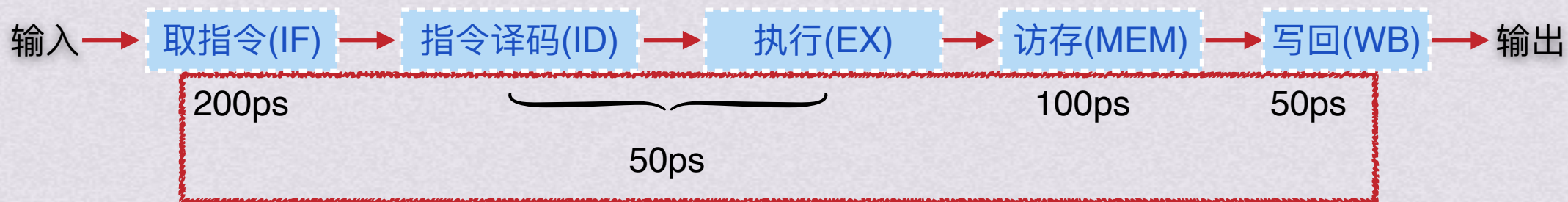
指令流水段个数以最复杂指令所用的功能段个数为准

流水段的长度以最复杂功能段的操作所用时间为准



# 指令流水线

## 流水段的基本构成



流水线的通路设计原则:

五段流水线

指令流水段个数以最复杂指令所用的功能段个数为准

流水段的长度以最复杂功能段的操作所用时间为准



# 指令流水线

## 流水段的基本构成



流水线的数据通路设计原则:

五段流水线

指令流水段个数以最复杂指令所用的功能段个数为准

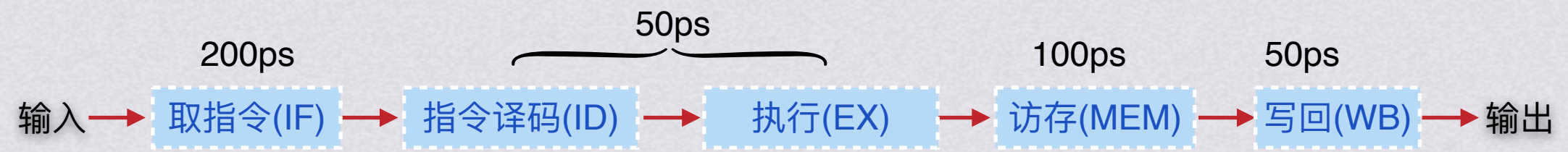
流水段的长度以最复杂功能段的操作所用时间为准

每段流水线长度200ps

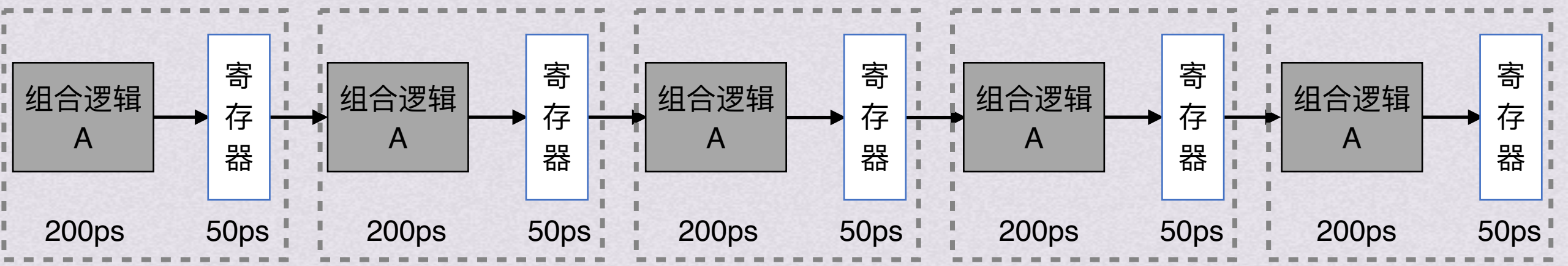


# 指令流水线

## 流水段的基本构成

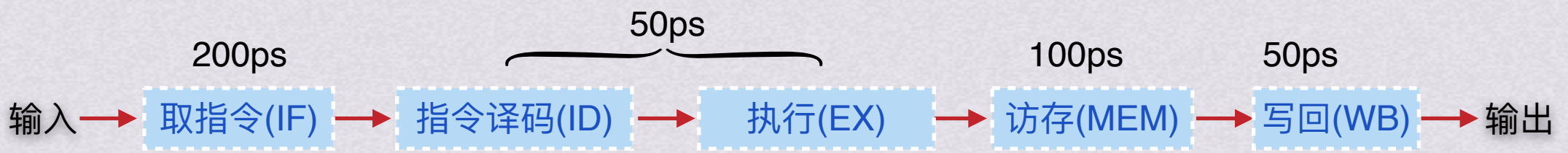


每个流水段包括一个组合逻辑和一个寄存器  
寄存器用来保存对应组合逻辑处理的结果

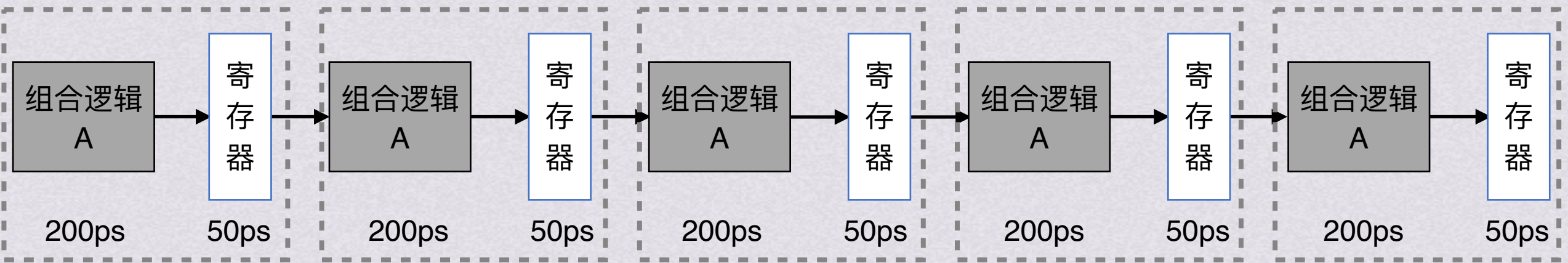


# 指令流水线

## 流水段的基本构成



每个流水段包括一个组合逻辑和一个寄存器  
寄存器用来保存对应组合逻辑处理的结果

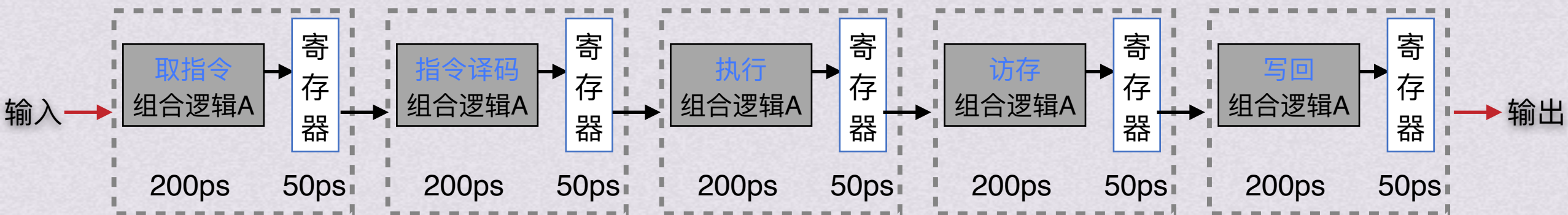


在不考虑控制单元、PC访问、信号传递等延迟的情况下  
时钟周期为200+50=250ps



## 指令流水线

## 流水段的基本构成



每个流水段包括一个组合逻辑和一个寄存器  
寄存器用来保存对应组合逻辑处理的结果

在不考虑控制单元、PC访问、信号传递等延迟的情况下  
时钟周期为 $200+50=250\text{ps}$



# 适合流水线的指令集特征



# 适合流水线的指令集特征

- 1.指令长度尽量一致
- 2.指令格式尽量规整
- 3.采用load/store型指令风格
- 4.数据和指令在存储器中要“对齐”存放



# 适合流水线的指令集特征

## 1.指令长度尽量一致

有利于简化取指令和指令译码操作

如果指令长度不等，取指令部件会变得更复杂，也不利于指令译码

2.指令格式尽量规整

3.采用load/store型指令风格

4.数据和指令在存储器中要“对齐”存放



# 适合流水线的指令集特征

1.指令长度尽量一致

## 2.指令格式尽量规整

尽量保证源操作数寄存器的位置相同，有利于在指令未译码时就可以读取寄存器操作数

3.采用load/store型指令风格

4.数据和指令在存储器中要“对齐”存放



# 适合流水线的指令集特征

1.指令长度尽量一致 2.指令格式尽量规整

## 3.采用load/store型指令风格

指的是指令集中只有load指令和store指令能访问存储器，其他指令一律不能访问

load用于将数据从存储单元读出并装入寄存器(如MIPS中的lw指令)

store指令用来把寄存器的内容写入存储单元

通过这种指令风格可以把load/store指令的地址计算和运算指令的执行步骤规整在同一个流水段中，有利于减少操作步骤，以规整流水线



# 适合流水线的指令集特征

1.指令长度尽量一致 2.指令格式尽量规整 3.采用load/store型指令风格

## 4.数据和指令在存储器中要“对齐”存放

能够使所需数据在一个流水段内就能从存储器中得到



# 指令流水线的实现



# 指令流水线的实现

假定分为以下  
五个流水线功能段

取指(Ifetch):组合逻辑A中有指令存储器,  
该阶段实现取指令功能

译码/取数(Reg/Dec):组合逻辑B中有寄存器堆,  
该阶段实现读寄存器内容并对指令译码

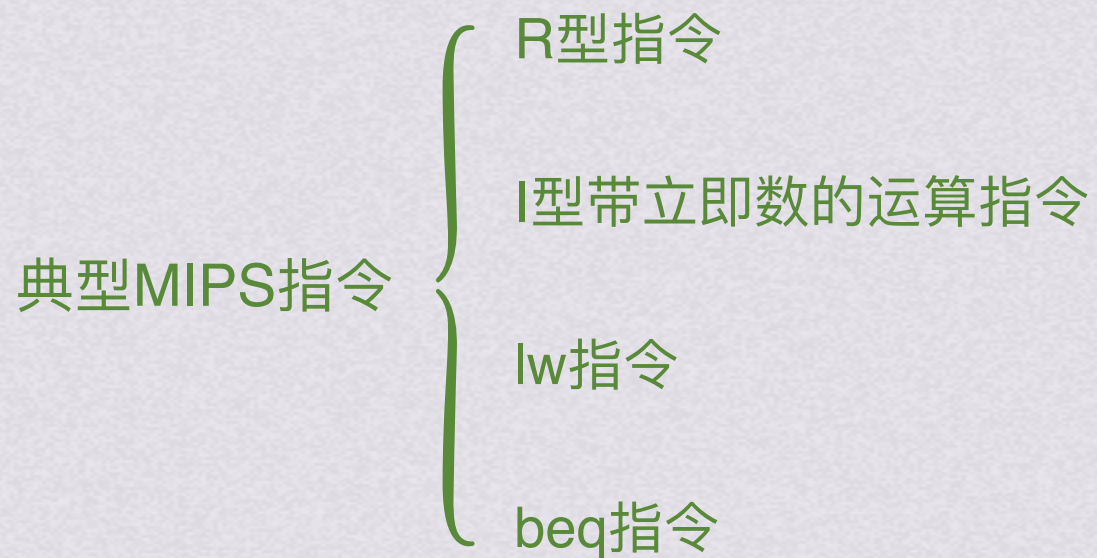
执行(ALU):组合逻辑C中有ALU,  
该阶段实现算数和逻辑运算操作

访存(Mem):组合逻辑D中有数据存储器,  
该阶段实现存储器读或写操作

写回(Write):组合逻辑E中有多路选择器,  
该阶段选择存入寄存器的结果并将其在寄存器输入端稳定保持一段建立时间



# 指令流水线的实现





# 指令流水线的实现典型MIPS指令

## R型指令

(汇编形式如“sub rd , rs , rt”)

涉及在ALU中对rs和rt内容的运算，并把ALU运算的结果送目的寄存器rd  
因此，在取指，译码/取数两个公共的功能段后，  
ALU功能段在ALU中计算；  
Mem功能段为空操作；  
Write功能段将ALU运算结果写寄存器rd



I型带立即数的运算指令

lw指令

beq指令



# 指令流水线的实现典型MIPS指令

## R型指令

(汇编形式如“sub rd , rs , rt”)

涉及在ALU中对rs和rt内容的运算，并把ALU运算的结果送目的寄存器rd  
因此，在取指，译码/取数两个公共的功能段后，  
ALU功能段在ALU中计算；  
Mem功能段为空操作；  
Write功能段将ALU运算结果写寄存器rd



I型带立即数的运算指令

lw指令

beq指令



# 指令流水线的实现

典型MIPS指令 R型指令

## I型带立即数的运算指令

(汇编形式如“addi rt , rs , imm16”)

涉及对16位立即数imm16进行符号扩展或零扩展，然后和rs的内容进行运算，最终把ALU的运算结果送目的寄存器rt

I型运算类指令的功能段划分与R型指令类似



lw指令

beq指令



# 指令流水线的实现

典型MIPS指令 R型指令

## I型带立即数的运算指令

(汇编形式如“addi rt , rs , imm16”)

涉及对16位立即数imm16进行符号扩展或零扩展，然后和rs的内容进行运算，最终把ALU的运算结果送目的寄存器rt

I型运算类指令的功能段划分与R型指令类似





# 指令流水线的实现

典型MIPS指令

R型指令

I型带立即数的运算指令

## lw指令

(汇编形式如“lw rt , imm16(rs)”)

功能为 $R[rt] \leftarrow M[R[rs] + \text{SignExt}(\text{imm16})]$  即把存储单元的内容装入寄存器中

除两个公共的功能段外,

ALU功能段在ALU中计算存储单元地址;

Mem功能段从存储单元中出数据;

Write功能段将数据写寄存器rt





# 指令流水线的实现

典型MIPS指令

R型指令 I型带立即数的运算指令

## lw指令

(汇编形式如“lw rt , imm16(rs)”)

功能为 $R[rt] \leftarrow M[R[rs] + \text{SignExt}(\text{imm16})]$  即把存储单元的内容装入寄存器中

除两个公共的功能段外,

ALU功能段在ALU中计算存储单元地址;

Mem功能段从存储单元中出数据;

Write功能段将数据写寄存器rt





# 指令流水线的实现

典型MIPS指令

R型指令

I型带立即数的运算指令

lw指令

## beq指令

(汇编形式如“beq rs , rt , imm16”)

是一条**分支指令（条件转移指令）**，指令的功能如下

```
if R[rs] = R[rt]
    PC ← PC + 4 + 4 x imm16
else
    PC ← PC + 4
```

除两个公共的功能段外，beq指令的ALU功能段对R[rs]和R[rt]做减法，同时计算转移目标地址 $PC+4+4 \times imm16$ ；  
Mem功能段根据R[rs]和R[rt]是否相等来确定是否将转移目标地址送PC；  
write功能段为空操作





# 指令流水线的实现

典型MIPS指令

R型指令

I型带立即数的运算指令

lw指令

## beq指令

(汇编形式如“beq rs , rt , imm16”)

是一条**分支指令（条件转移指令）**，指令的功能如下

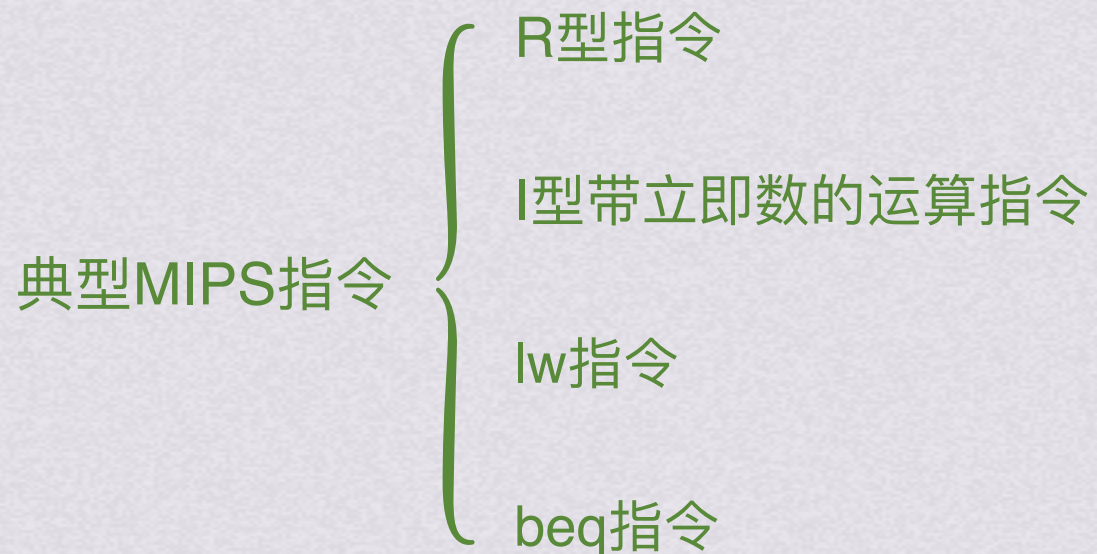
```
if R[rs] = R[rt]
    PC ← PC + 4 + 4 x imm16
else
    PC ← PC + 4
```

除两个公共的功能段外，beq指令的ALU功能段对R[rs]和R[rt]做减法，同时计算转移目标地址 $PC+4+4 \times imm16$ ；  
Mem功能段根据R[rs]和R[rt]是否相等来确定是否将转移目标地址送PC；  
write功能段为空操作





# 指令流水线的实现



在实际执行过程中，会遇见一些情况，使流水线无法正确、按时执行后序指令，从而引起**流水线阻塞**或**停顿**  
我们称这种现象为**流水线冒险**

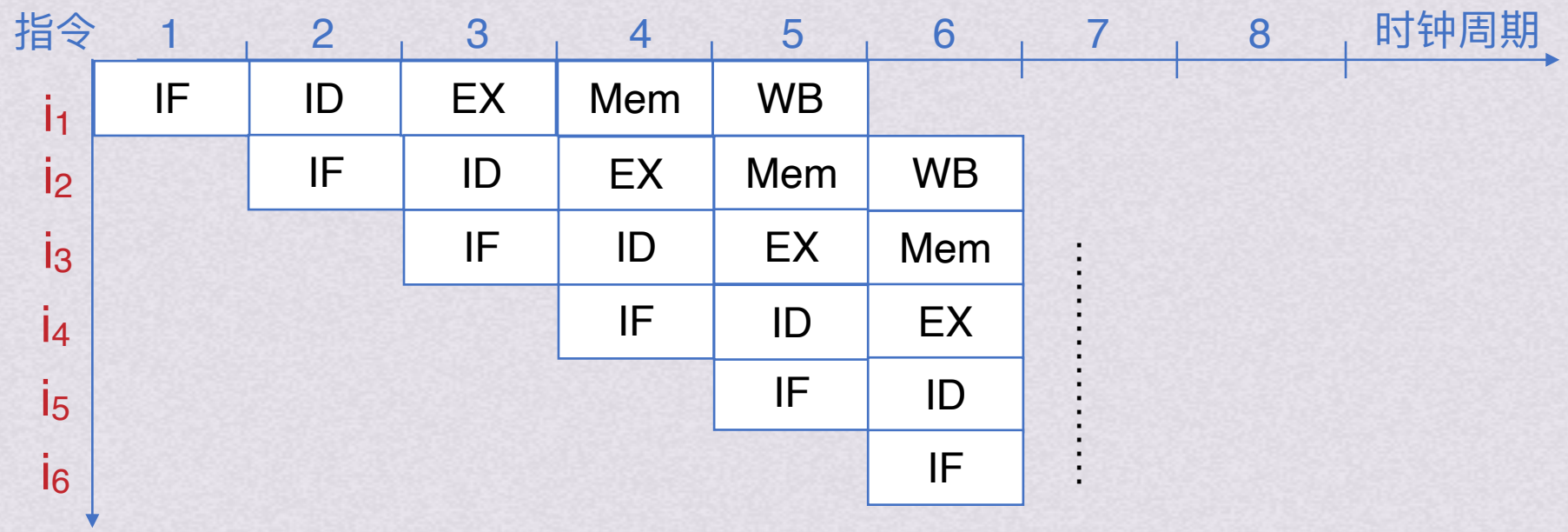
# 指令流水线的实现 流水线冒险

## 结构冒险

(硬件资源冲突)

同一个部件同时被不同指令所用，也就是硬件资源竞争造成的

若不区分指令存储器和数据存储器而只用一个存储器



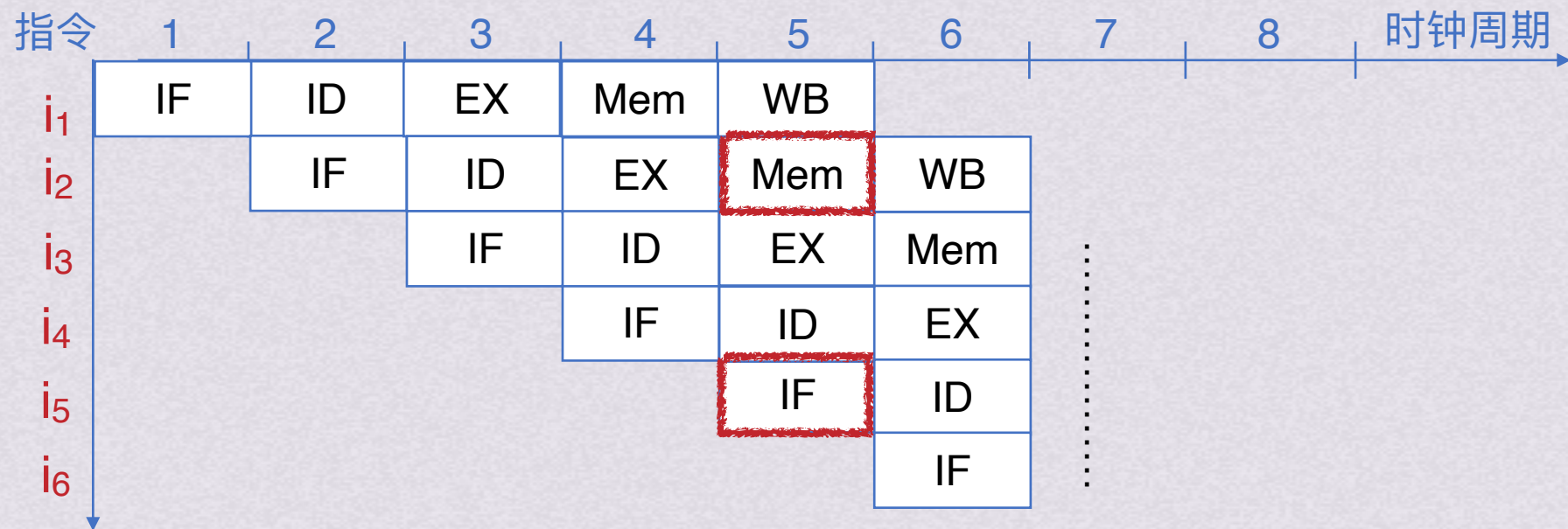
# 指令流水线的实现 流水线冒险

## 结构冒险

(硬件资源冲突)

同一个部件同时被不同指令所用，也就是硬件资源竞争造成的

若不区分指令存储器和数据存储器而只用一个存储器



两条指令会访问同一个存储器，发生访存冲突，引起结构冒险



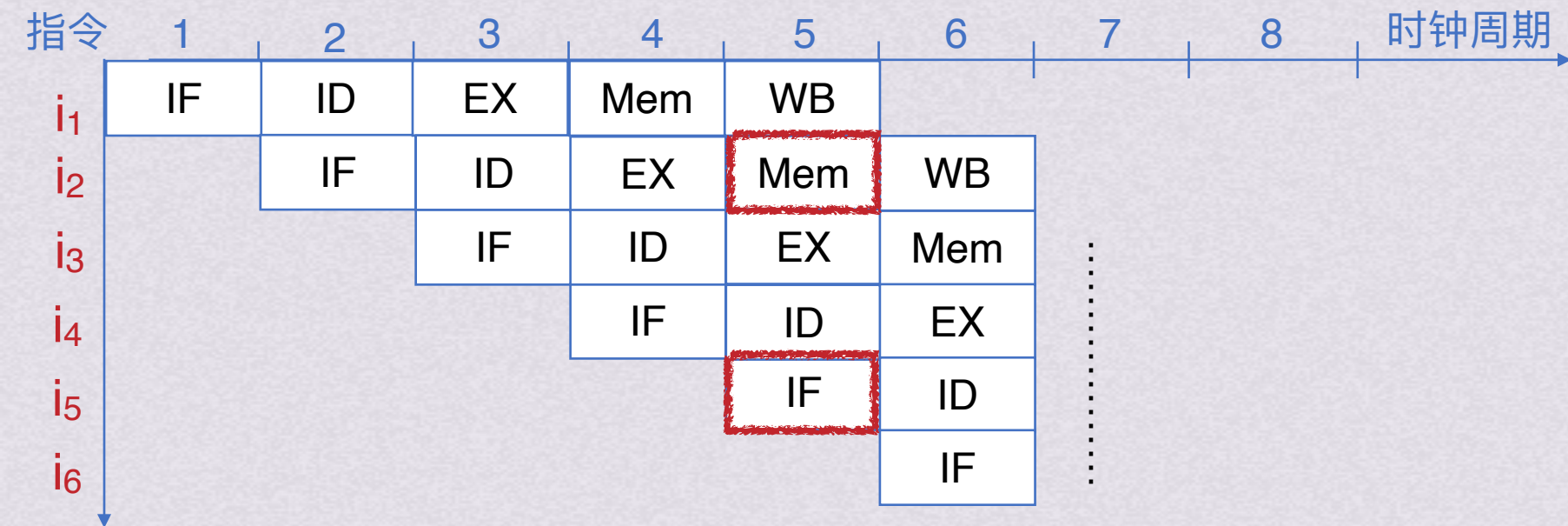
# 指令流水线的实现 流水线冒险

## 结构冒险

(硬件资源冲突)

同一个部件同时被不同指令所用，也就是硬件资源竞争造成的

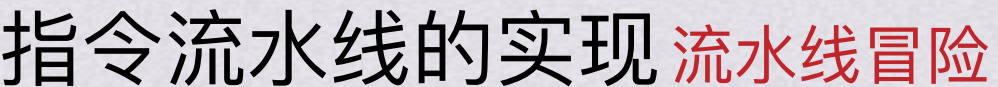
若不区分指令存储器和数据存储器而只用一个存储器



### 解决策略

- 1.规定一个部件每条指令只能使用一次，只能在特定阶段使用
- 2.通过设置多个独立的部件避免资源冲突（本例可以把指令存储器和数据存储器分开）

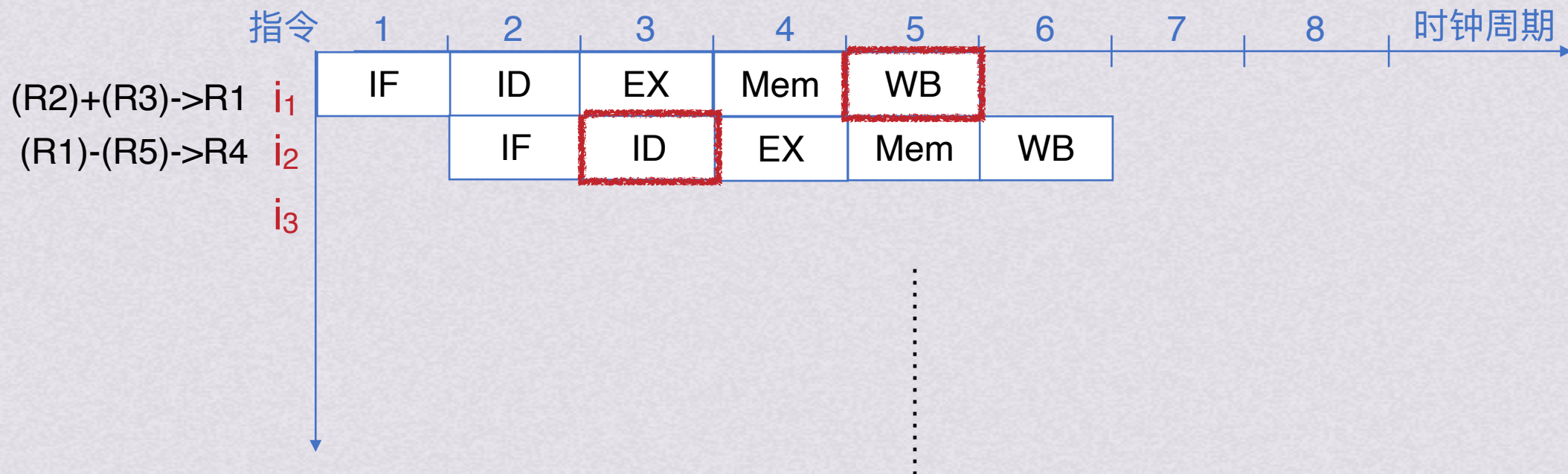
两条指令会访问同一个存储器，发生访存冲突，引起结构冒险

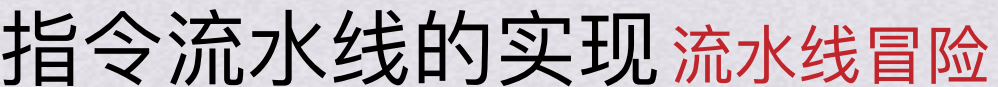


(也叫数据相关)

解决策略:由编译器在数据相关的指令之间加nop（空操作）指令，硬件阻塞等

2.采用数据转发机制，数据通路中一旦产生运算结果或一旦存储器读出数据，就把他们通过一条旁路直接送到相关后续指令在ALU阶段的ALU输入端





(也叫数据相关)

解决策略: 1.由编译器在数据相关的指令之间加nop (空操作) 指令, 硬件阻塞等

[illegible]



# 指令流水线的实现

流水线冒险

## 数据冒险

(也叫数据相关)

后面指令用到前面指令的运算结果时，前面指令的结果还没产生

- 解决策略:
- 1.由编译器在数据相关的指令之间加nop（空操作）指令，硬件阻塞等
  - 2.采用数据转发机制，数据通路中一旦产生运算结果或一旦存储器读出数据，就把他们通过一条旁路直接送到相关后续指令在ALU阶段的ALU输入端



# 指令流水线的实现

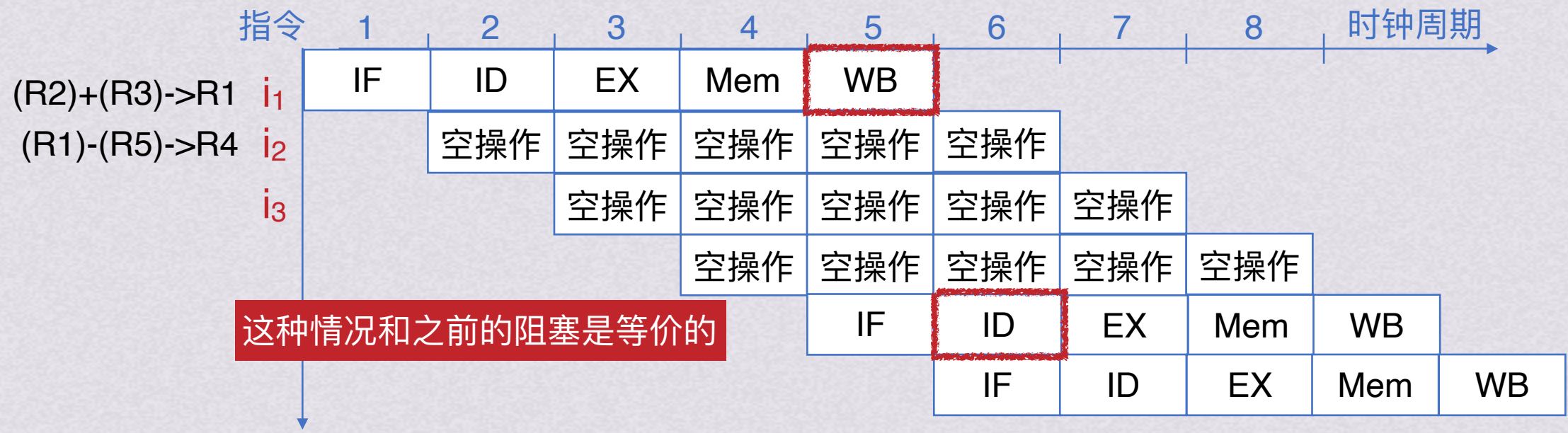
## 流水线冒险

## 数据冒险

(也叫数据相关)

后面指令用到前面指令的运算结果时，前面指令的结果还没产生

- 解决策略:
- 1.由编译器在数据相关的指令之间加nop（空操作）指令，硬件阻塞等
  - 2.采用数据转发机制，数据通路中一旦产生运算结果或一旦存储器读出数据，就把他们通过一条旁路直接送到相关后续指令在ALU阶段的ALU输入端



# 指令流水线的实现

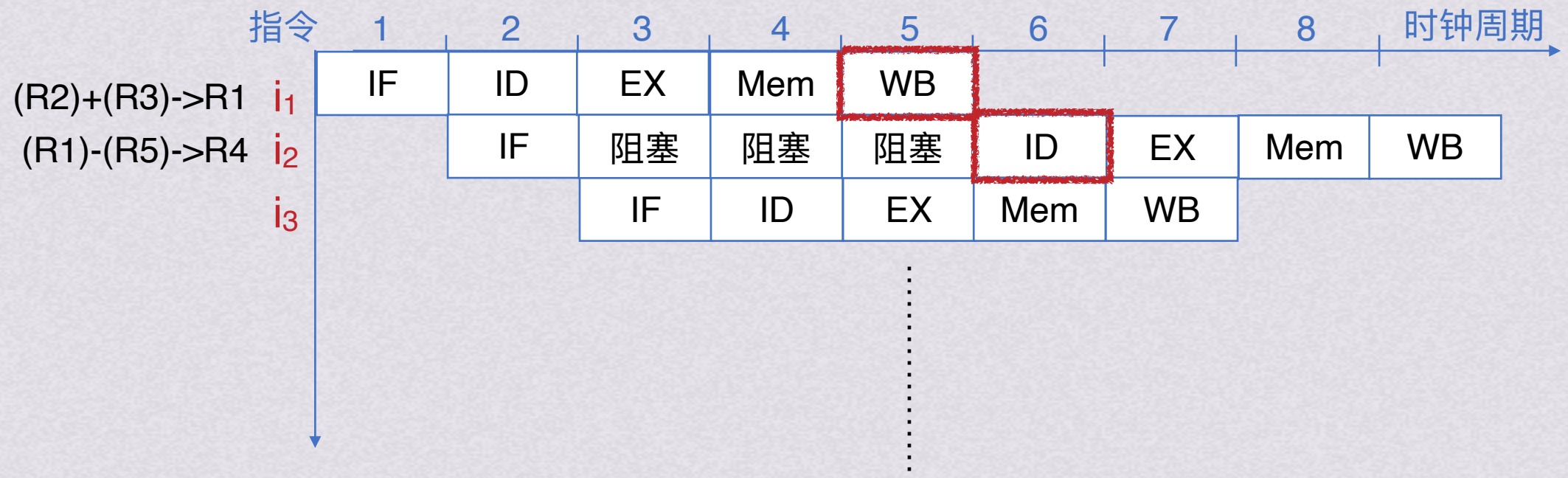
## 流水线冒险

## 数据冒险

(也叫数据相关)

后面指令用到前面指令的运算结果时，前面指令的结果还没产生

- 解决策略:
- 1.由编译器在数据相关的指令之间加nop（空操作）指令，硬件阻塞等
  - 2.采用数据转发机制，数据通路中一旦产生运算结果或一旦存储器读出数据，就把他们通过一条旁路直接送到相关后续指令在ALU阶段的ALU输入端



# 指令流水线的实现

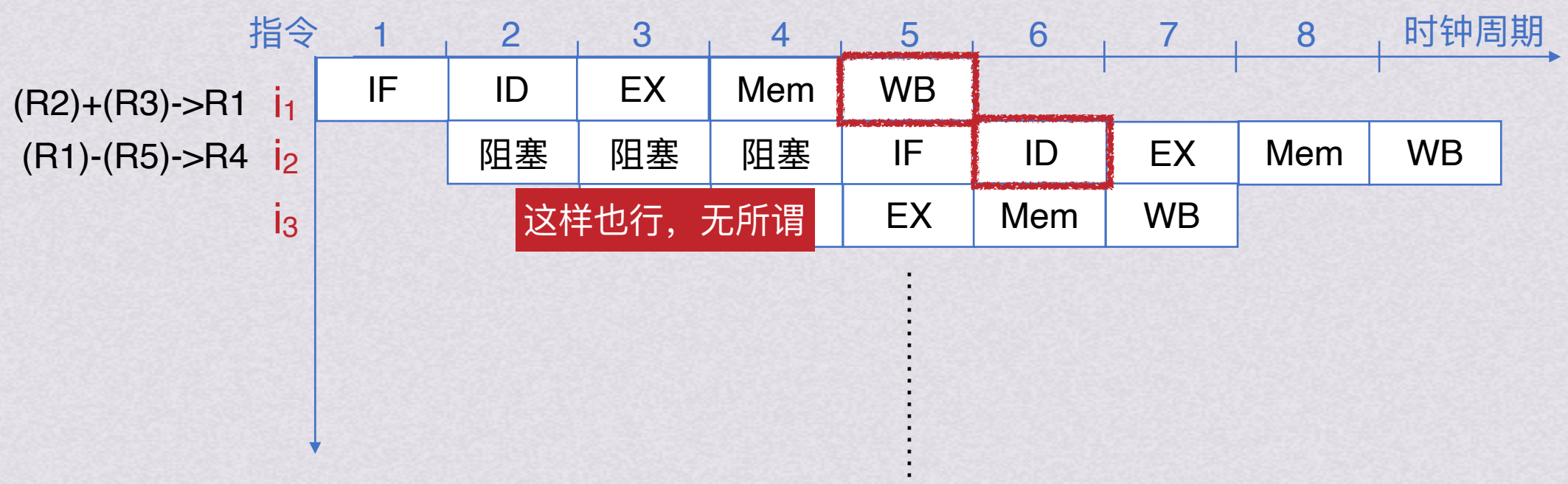
## 流水线冒险

## 数据冒险

(也叫数据相关)

后面指令用到前面指令的运算结果时，前面指令的结果还没产生

- 解决策略:
- 1.由编译器在数据相关的指令之间加nop（空操作）指令，硬件阻塞等
  - 2.采用数据转发机制，数据通路中一旦产生运算结果或一旦存储器读出数据，就把他们通过一条旁路直接送到相关后续指令在ALU阶段的ALU输入端



# 指令流水线的实现

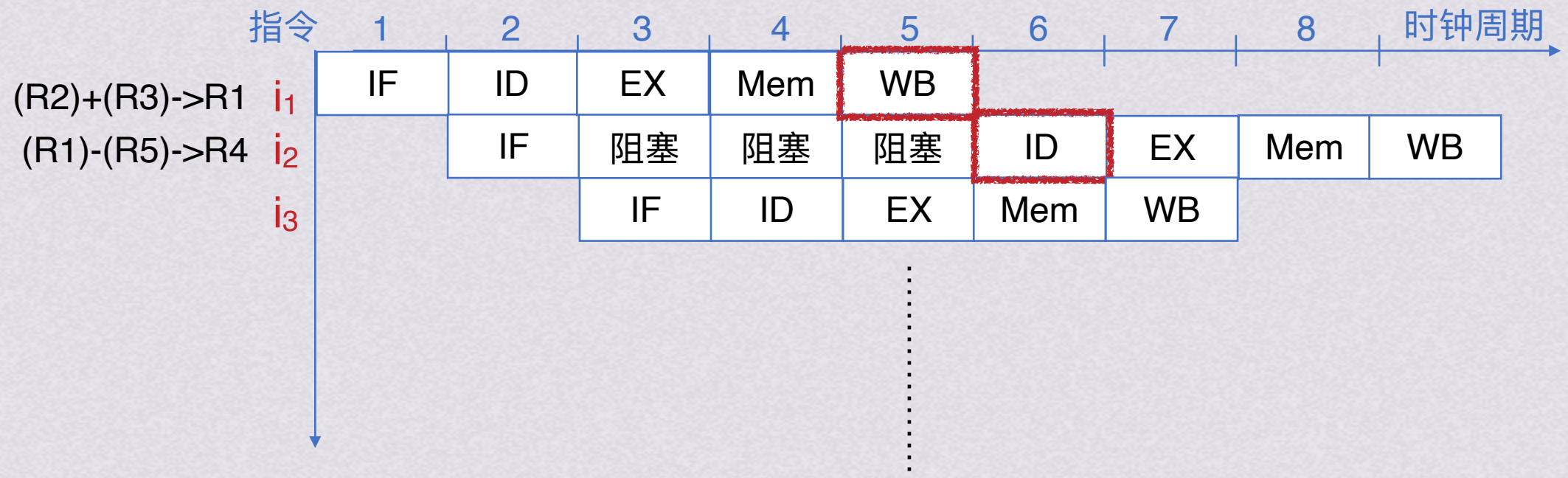
## 流水线冒险

## 数据冒险

(也叫数据相关)

后面指令用到前面指令的运算结果时，前面指令的结果还没产生

- 解决策略:
- 1.由编译器在数据相关的指令之间加nop（空操作）指令，硬件阻塞等
  - 2.采用数据转发机制，数据通路中一旦产生运算结果或一旦存储器读出数据，就把他们通过一条旁路直接送到相关后续指令在ALU阶段的ALU输入端



# 指令流水线的实现

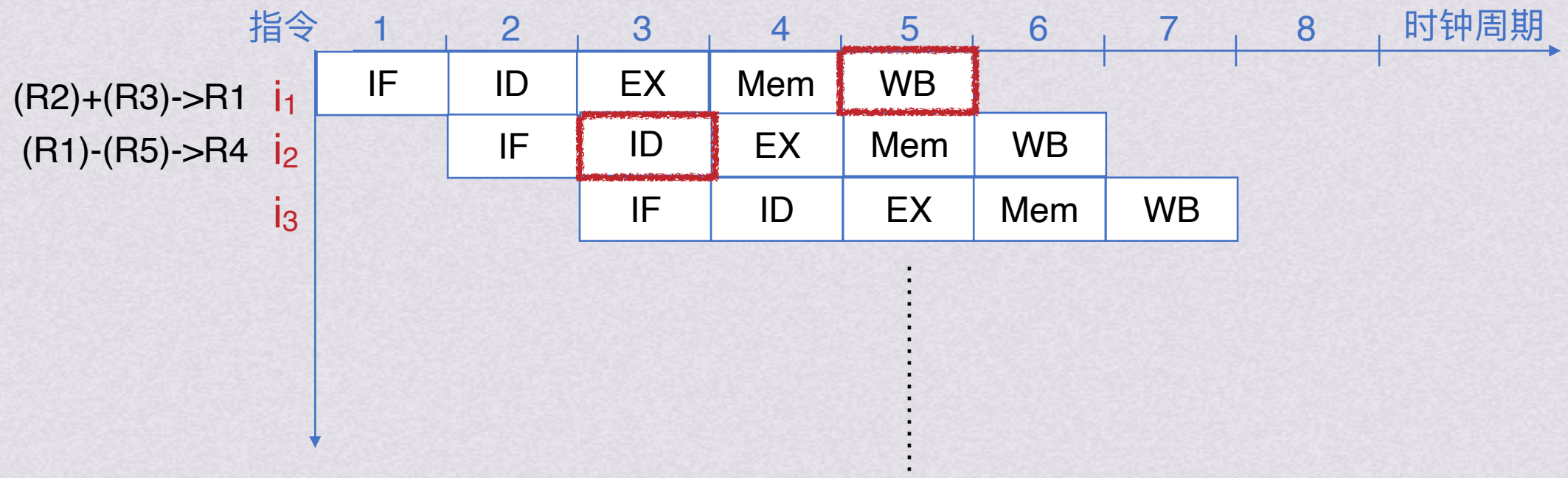
## 流水线冒险

## 数据冒险

(也叫数据相关)

后面指令用到前面指令的运算结果时，前面指令的结果还没产生

- 解决策略:
- 1.由编译器在数据相关的指令之间加nop（空操作）指令，硬件阻塞等
  - 2.采用数据转发机制，数据通路中一旦产生运算结果或一旦存储器读出数据，就把他们通过一条旁路直接送到相关后续指令在ALU阶段的ALU输入端



# 指令流水线的实现

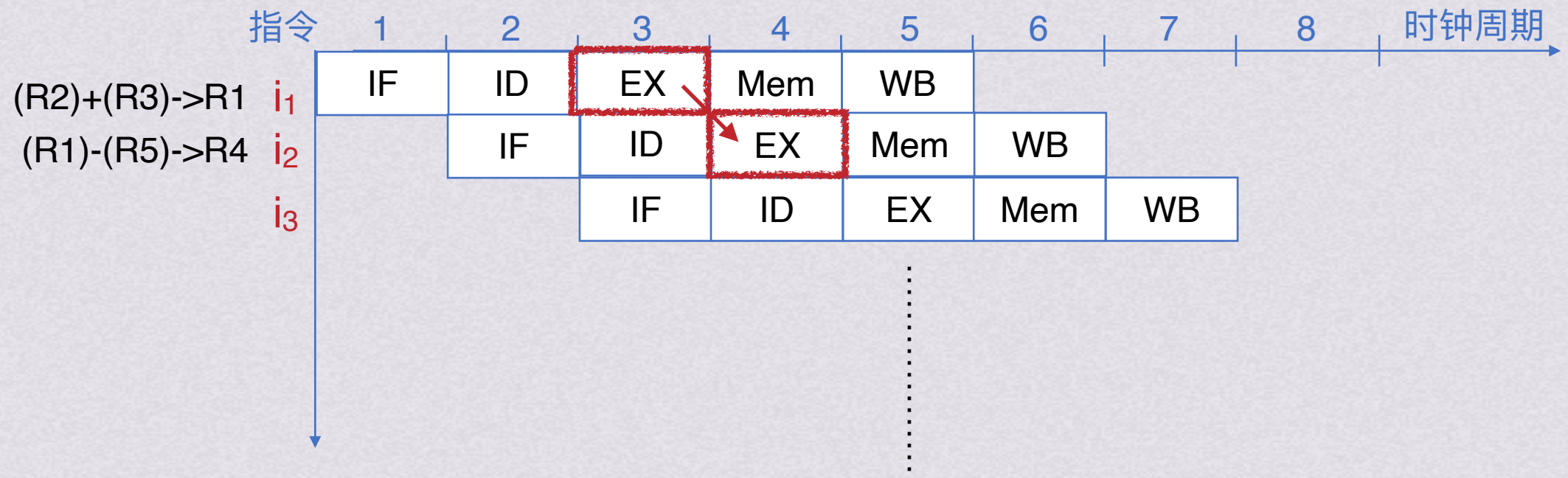
## 流水线冒险

## 数据冒险

(也叫数据相关)

后面指令用到前面指令的运算结果时，前面指令的结果还没产生

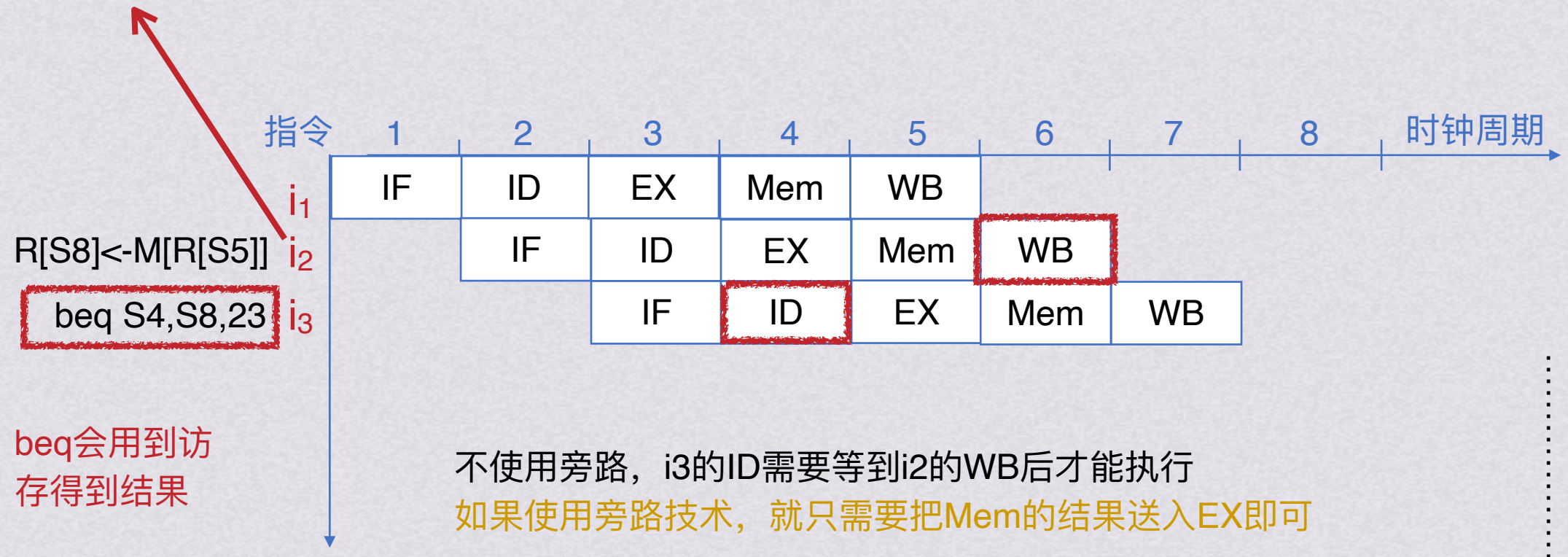
- 解决策略:
- 1.由编译器在数据相关的指令之间加nop（空操作）指令，硬件阻塞等
  - 2.采用数据转发机制，数据通路中一旦产生运算结果或一旦存储器读出数据，就把他们通过一条旁路直接送到相关后续指令在ALU阶段的ALU输入端



# load-use冒险

前一条指令为访存并将数据存储到寄存器  
后一条指令需要使用该数据

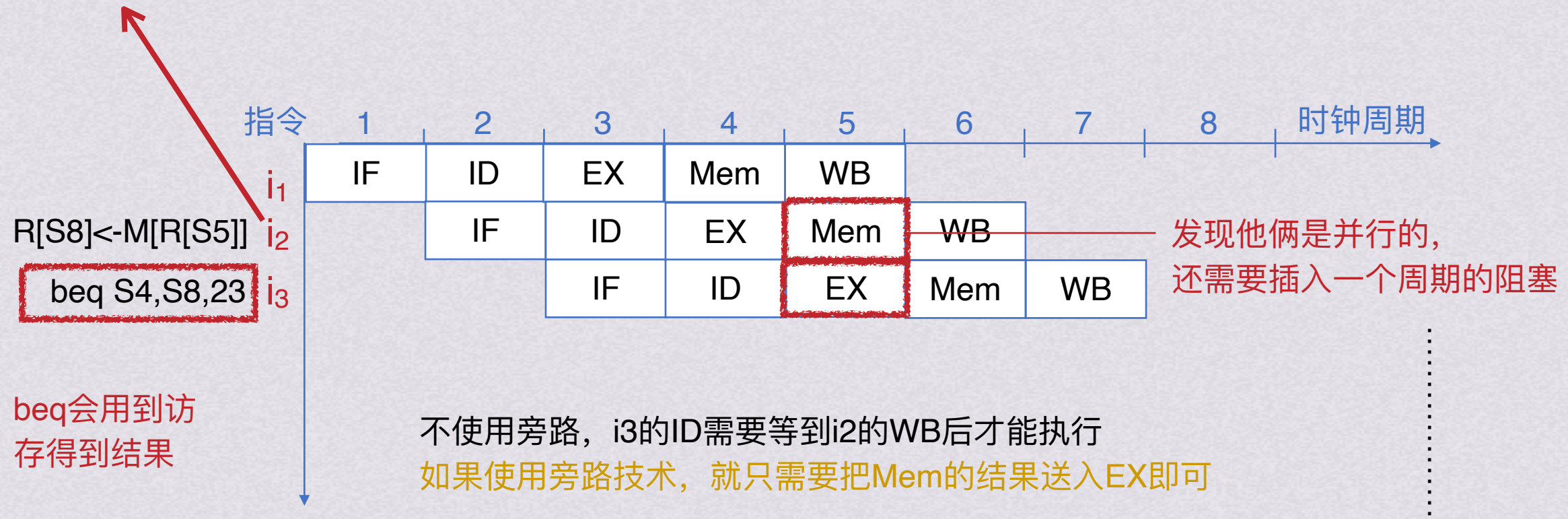
按照5号寄存器内容访存并  
且写入结果到8号寄存器



# load-use冒险

前一条指令为访存并将数据存储到寄存器  
后一条指令需要使用该数据

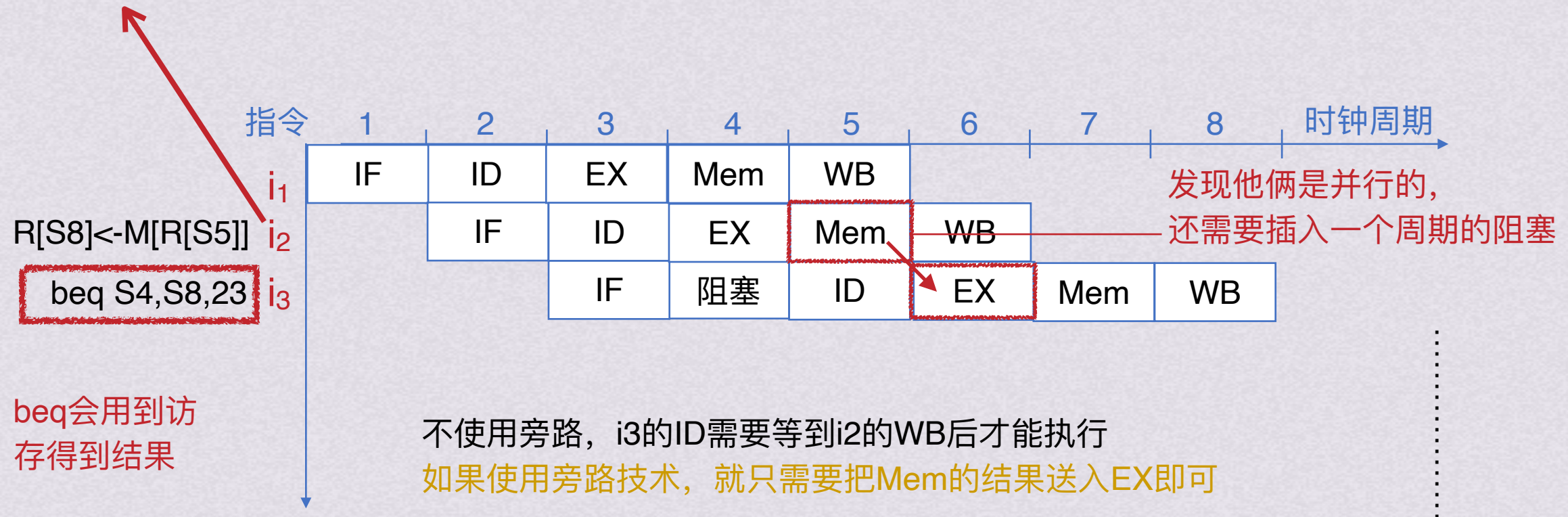
按照5号寄存器内容访存并  
且写入结果到8号寄存器



# load-use冒险

前一条指令为访存并将数据存储到寄存器  
后一条指令需要使用该数据

按照5号寄存器内容访存并  
且写入结果到8号寄存器



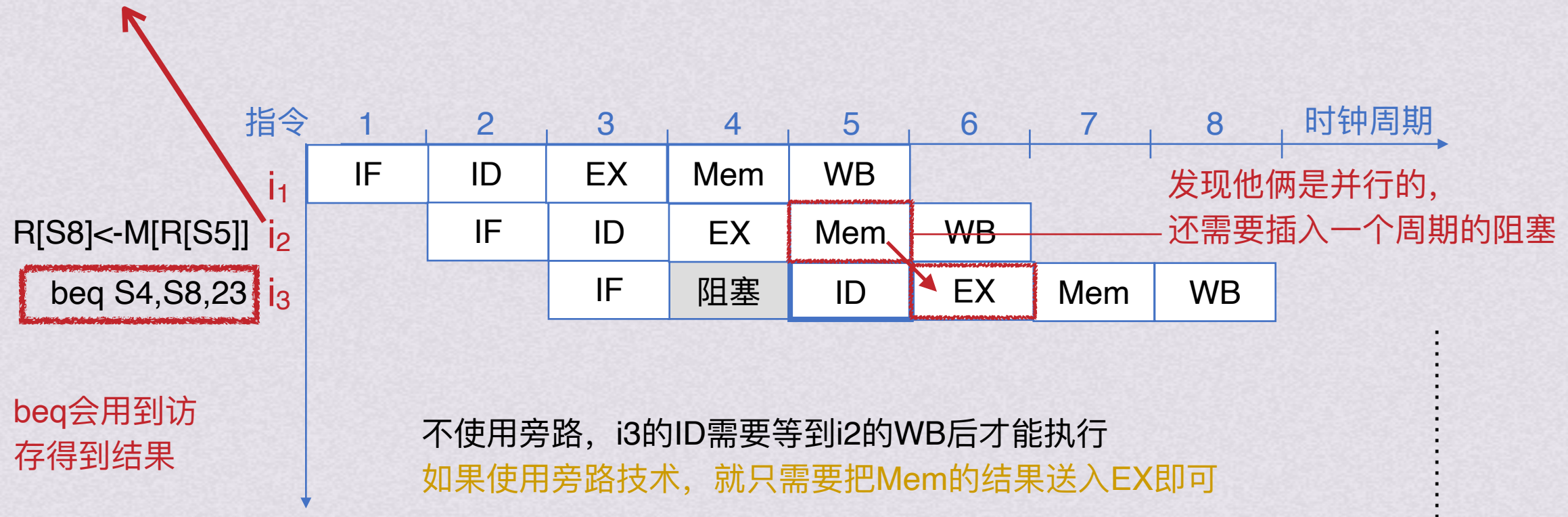
不使用旁路, i<sub>3</sub>的ID需要等到i<sub>2</sub>的WB后才能执行  
如果使用旁路技术, 就只需要把Mem的结果送入EX即可

load-use冒险在ID阶段进行检测, 检测到后必须使Load后的两条指令停顿一个时钟周期后继续执行

# load-use冒险

前一条指令为访存并将数据存储到寄存器  
后一条指令需要使用该数据

按照5号寄存器内容访存并  
且写入结果到8号寄存器



load-use冒险在ID阶段进行检测，检测到后必须使Load后的两条指令停顿一个时钟周期后继续执行



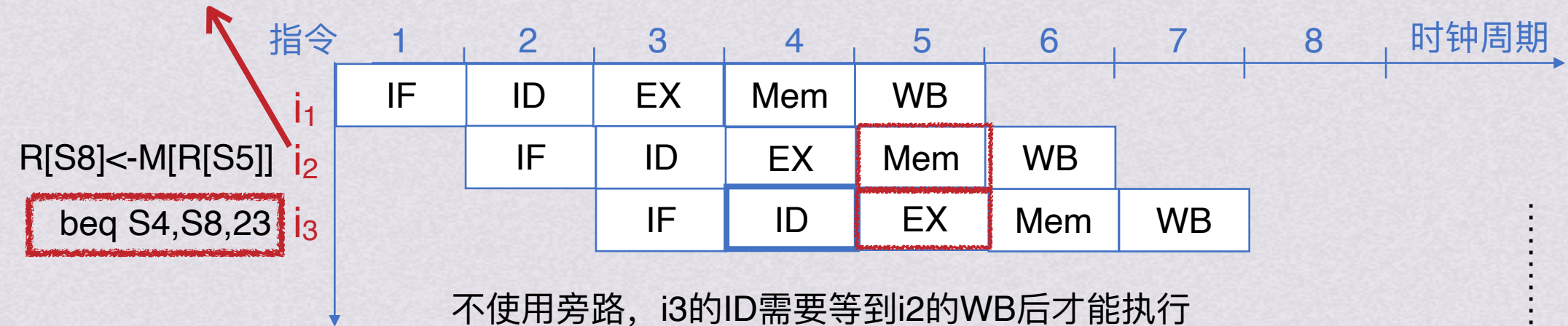
流水线冒险

数据冒险

# load-use冒险

按照5号寄存器内容访存并  
且写入结果到8号寄存器

前一条指令为访存并将数据存储到寄存器  
后一条指令需要使用该数据



$R[S8] \leftarrow M[R[S5]]$

beq S4,S8,23

beq会用到访  
存得到结果

不使用旁路， $i_3$ 的ID需要等到 $i_2$ 的WB后才能执行

如果使用旁路技术，就只需要把Mem的结果送入EX即可

load-use冒险在ID阶段进行检测，检测到后必须使Load后的两条指令停顿一个时钟周期后继续执行

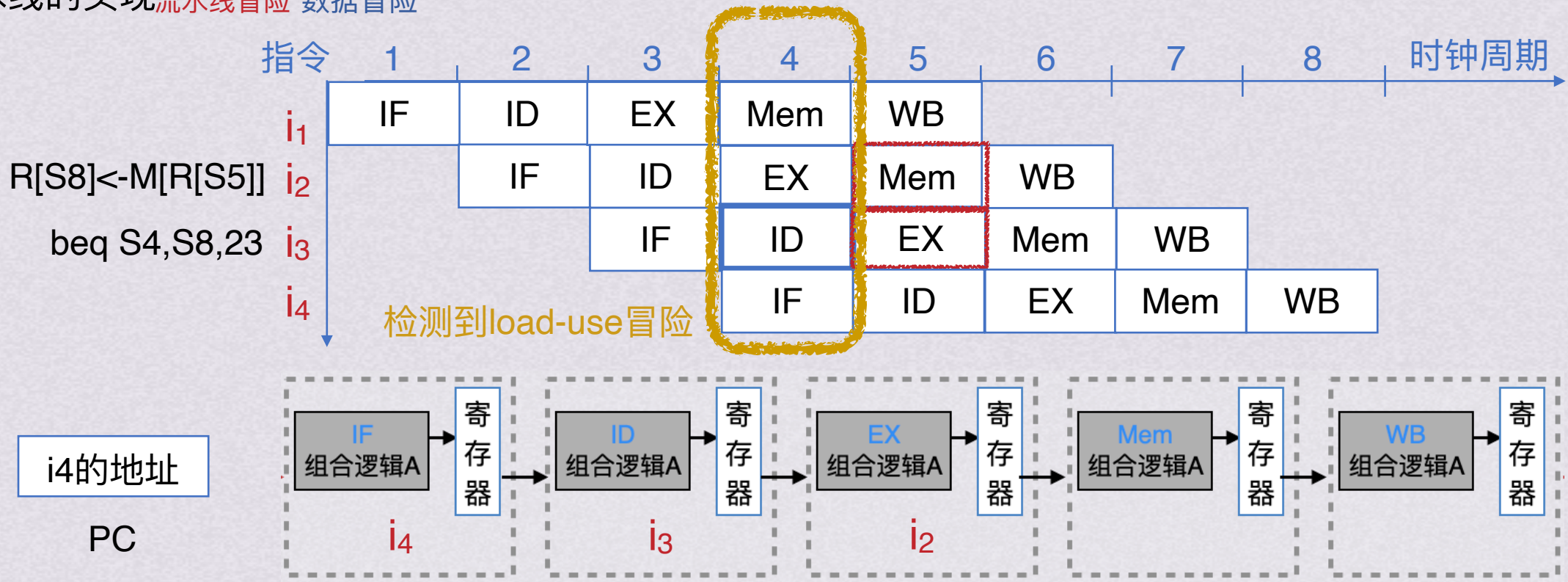
检测到冒险后会做三件事情：

- 1.把ID/EX流水段的控制信号清零，而不是把译码得到的控制信号给EX
- 2.保持IF/ID流水段寄存器的值不变，而不送入当时取出的指令
- 3.保持PC值不变，这样阻塞一个时钟周期后load的下下一条指令可以正常再取指一次



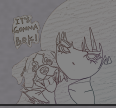
指令流水线的实现

流水线冒险 数据冒险



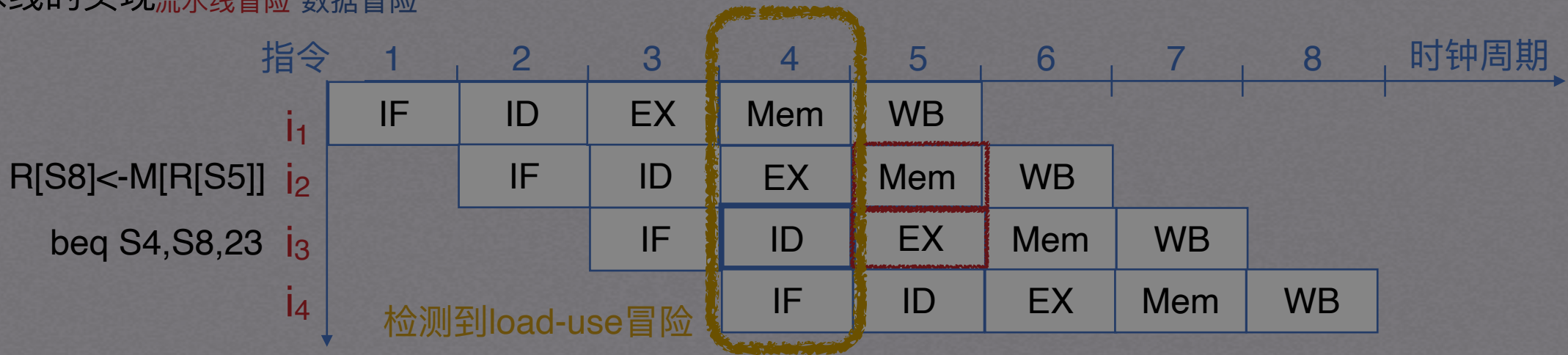
load-use冒险在ID阶段进行检测，检测到后必须使Load后的两条指令停顿一个时钟周期后继续执行

- 检测到冒险后会做三件事情:
- 1. 把ID/EX流水段的控制信号清零，而不是把译码得到的控制信号给EX
  - 2. 保持IF/ID流水段寄存器的值不变，而不送入当时取出的指令
  - 3. 保持PC值不变，这样阻塞一个时钟周期后load的下下一条指令可以正常再取指一次



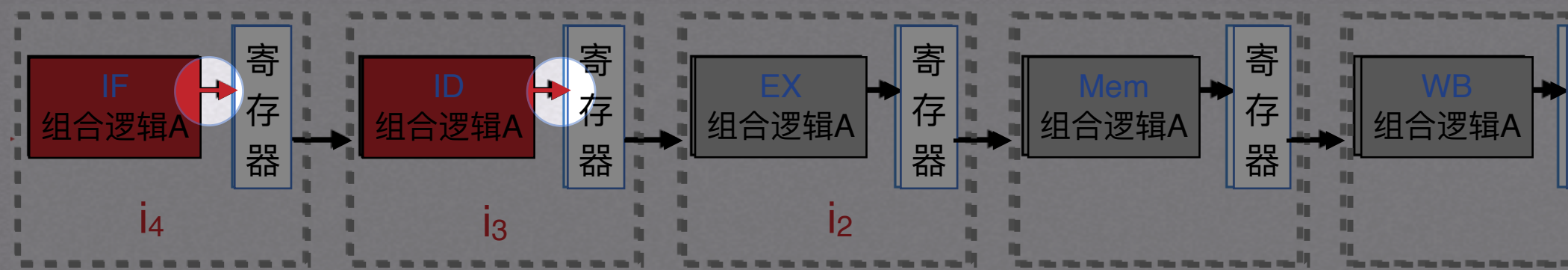
指令流水线的实现

流水线冒险 数据冒险



$i_4$ 的地址

PC

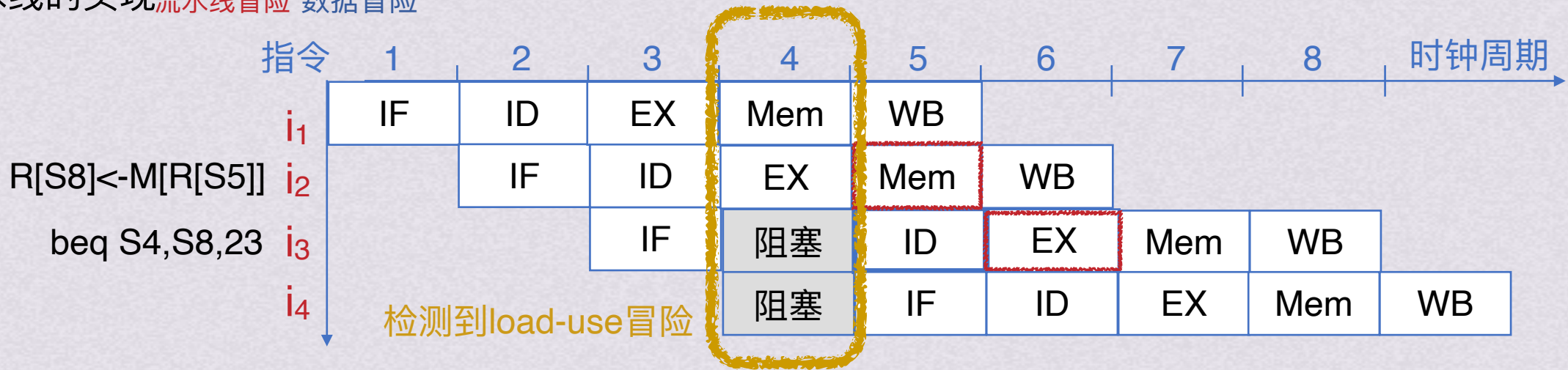


- 检测到冒险后会做三件事情：
- 1. 把ID/EX流水段的控制信号清零，而不是把译码得到的控制信号给EX
  - 2. 保持IF/ID流水段寄存器的值不变，而不送入当时取出的指令
  - 3. 保持PC值不变，这样阻塞一个时钟周期后load的下下一条指令可以正常再取指一次



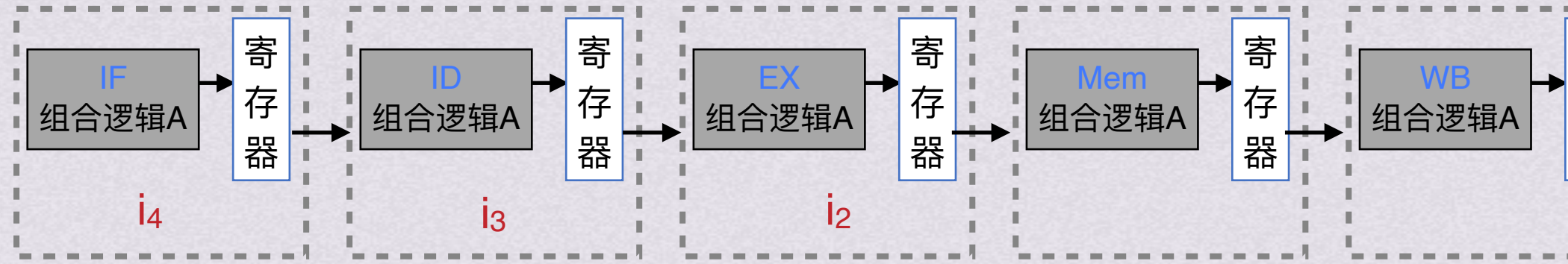
指令流水线的实现

流水线冒险 数据冒险



i4的地址

PC



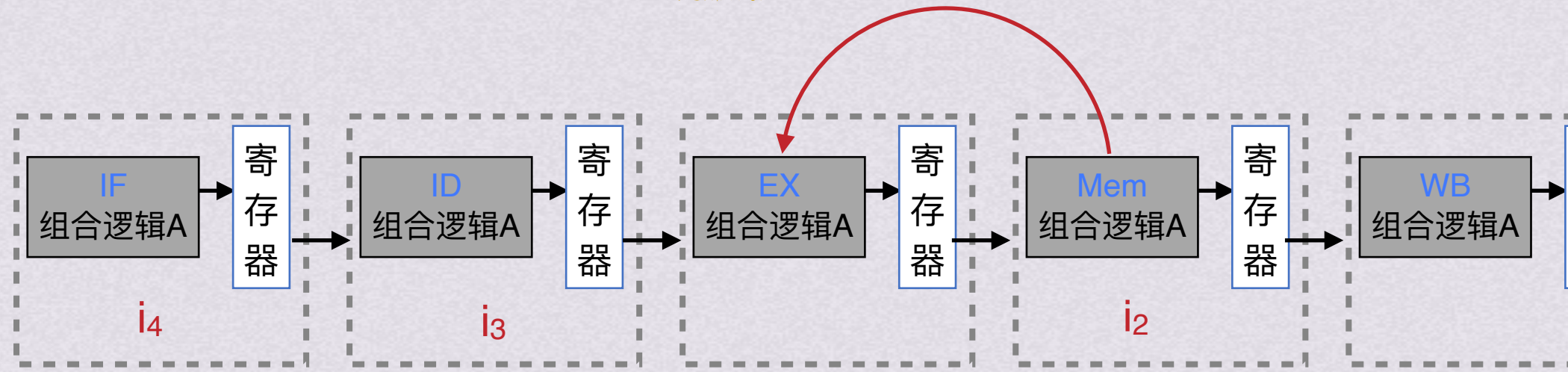
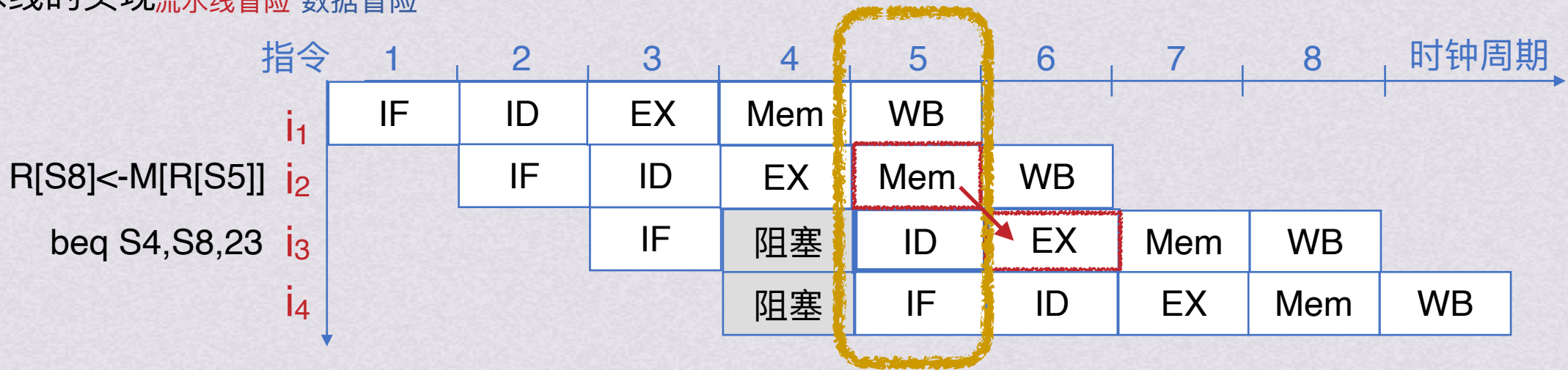
检测到冒险后会做三件事情:

- 1.把ID/EX流水段的控制信号清零，而不是把译码得到的控制信号给EX
- 2.保持IF/ID流水段寄存器的值不变，而不送入当时取出的指令
- 3.保持PC值不变，这样阻塞一个时钟周期后load的下下一条指令可以正常再取指一次



指令流水线的实现

流水线冒险 数据冒险

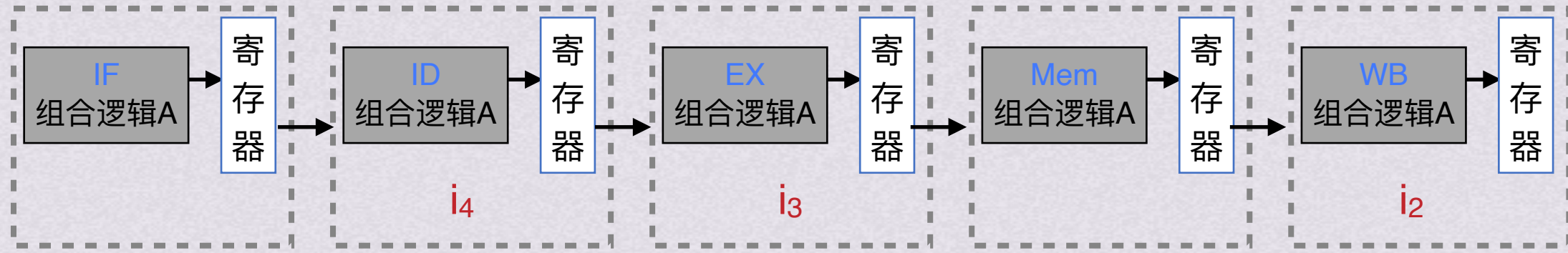
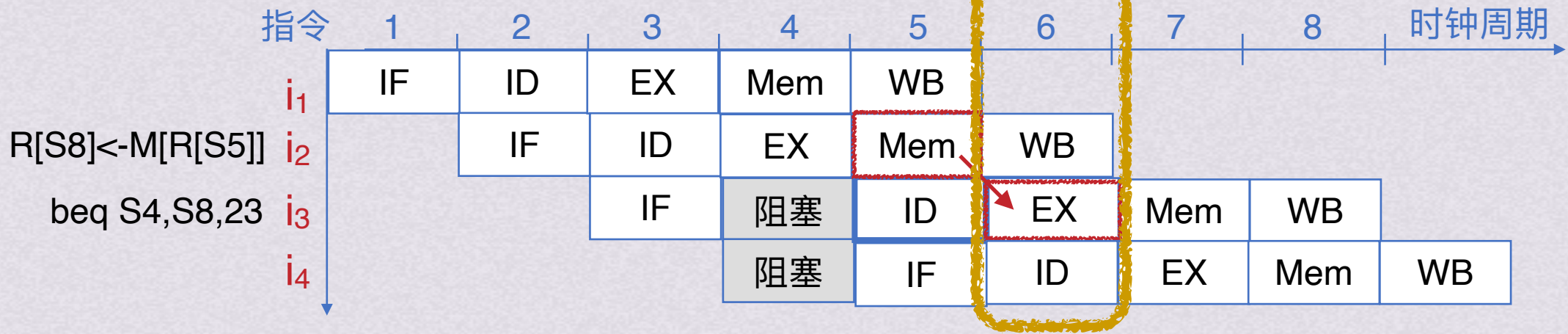


- 检测到冒险后会做三件事情：
- 1. 把ID/EX流水段的控制信号清零，而不是把译码得到的控制信号给EX
  - 2. 保持IF/ID流水段寄存器的值不变，而不送入当时取出的指令
  - 3. 保持PC值不变，这样阻塞一个时钟周期后load的下下一条指令可以正常再取指一次



指令流水线的实现

流水线冒险 数据冒险



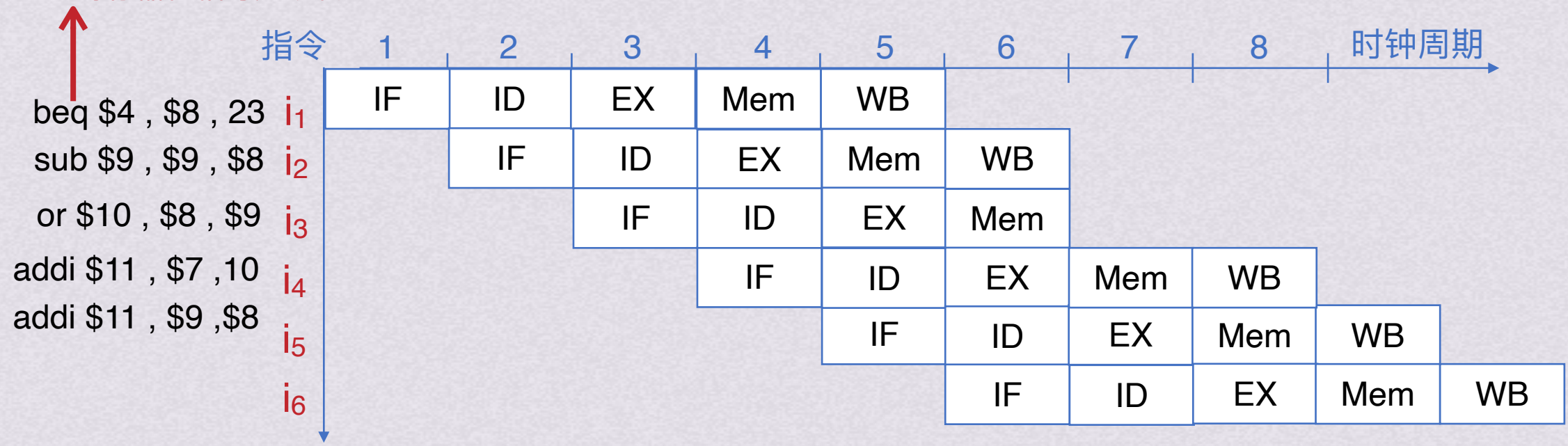
- 检测到冒险后会做三件事情：
- 1. 把ID/EX流水段的控制信号清零，而不是把译码得到的控制信号给EX
  - 2. 保持IF/ID流水段寄存器的值不变，而不送入当时取出的指令
  - 3. 保持PC值不变，这样阻塞一个时钟周期后load的下下一条指令可以正常再取指一次

# 指令流水线的实现 流水线冒险

## 控制冒险

遇到改变指令执行顺序的情况时，流水线中指令的正常执行会被阻塞，称控制冒险

若4号8号寄存器值相等则跳转



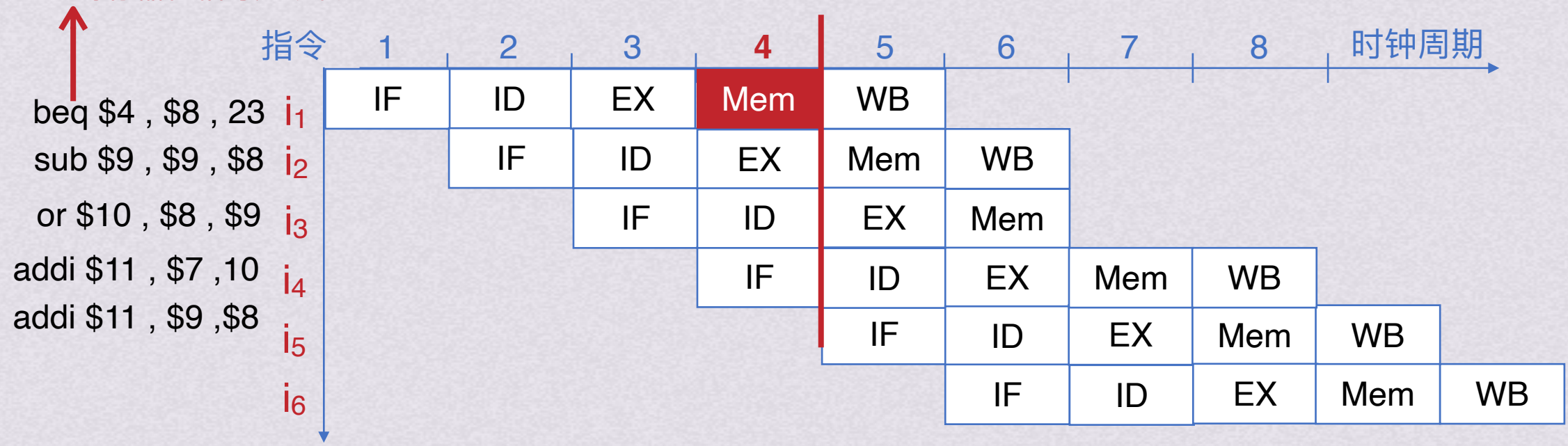
# 指令流水线的实现 流水线冒险

## 控制冒险

遇到改变指令执行顺序的情况时，流水线中指令的正常执行会被阻塞，称控制冒险

若4号8号寄存器值相等则跳转

该阶段根据是否相等确定是否将转移目标地址送PC



# 指令流水线的实现 流水线冒险

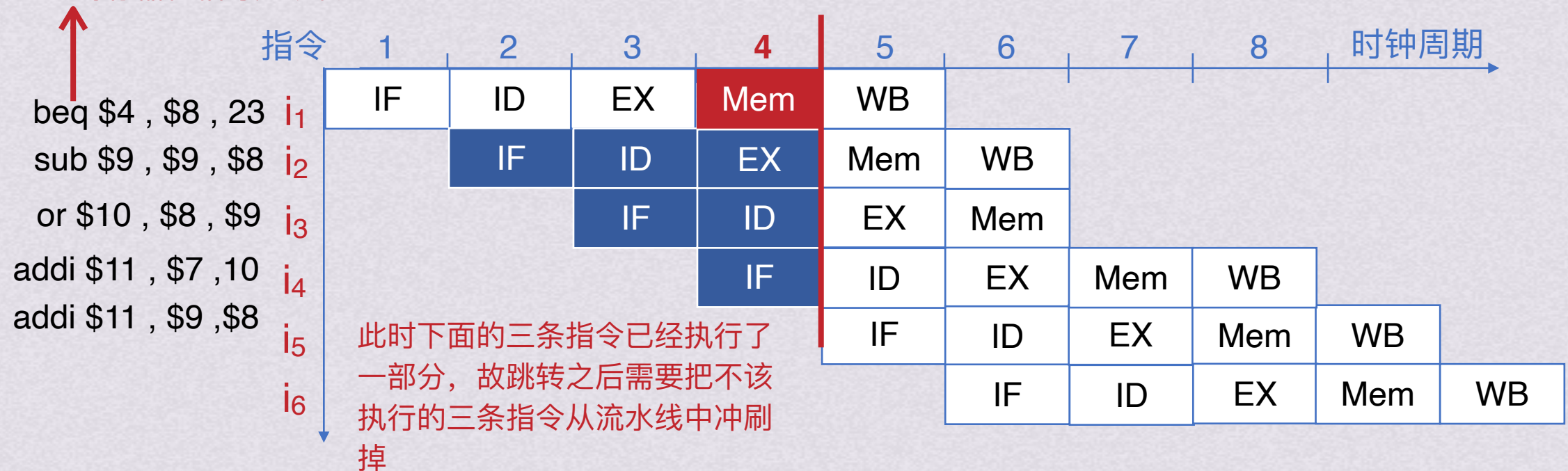
## 控制冒险

遇到改变指令执行顺序的情况时，流水线中指令的正常执行会被阻塞，称控制冒险

由于分支指令而引起的控制冒险也称**分支冒险**

若4号8号寄存器值相等则跳转

该阶段根据是否相等确定是否将转移目标地址送PC



此时下面的三条指令已经执行了一部分，故跳转之后需要把不该执行的三条指令从流水线中冲刷掉

将由于流水线阻塞而带来的延迟执行周期数量称为**延迟损失时间片**



# 流水线中的多发技术



# 流水线中的多发技术

超标量技术

超流水线技术

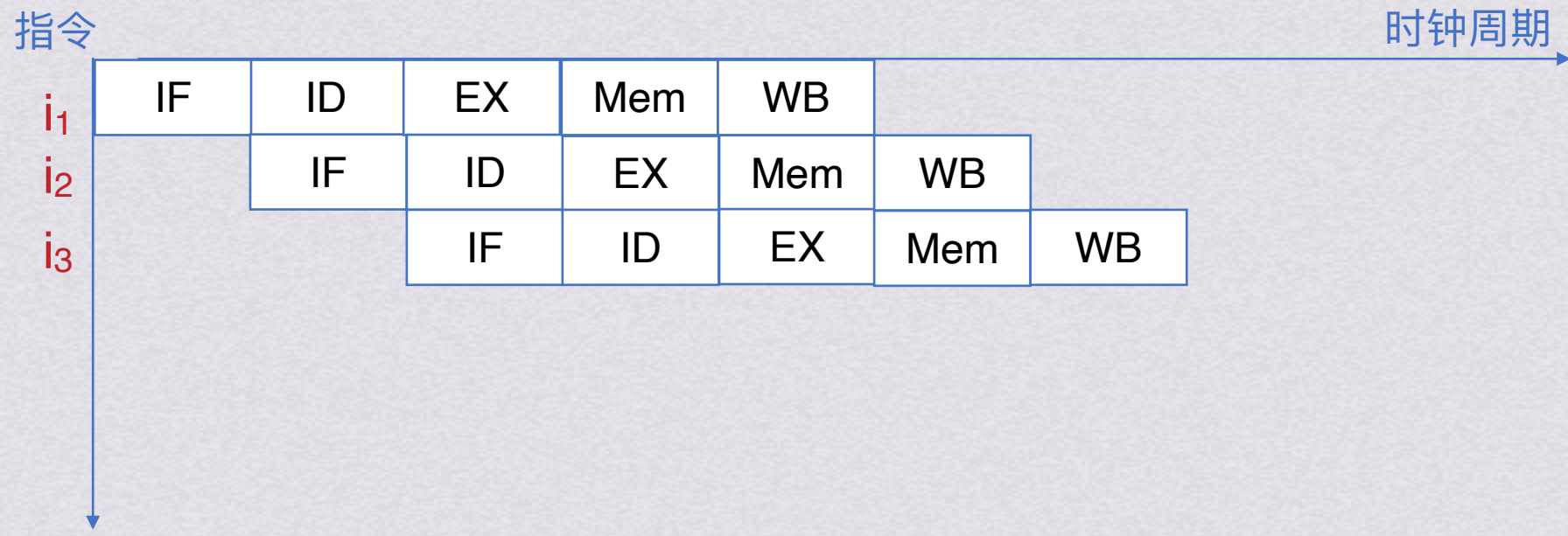
超长指令字技术



# 流水线中的多发技术

## 超标量技术

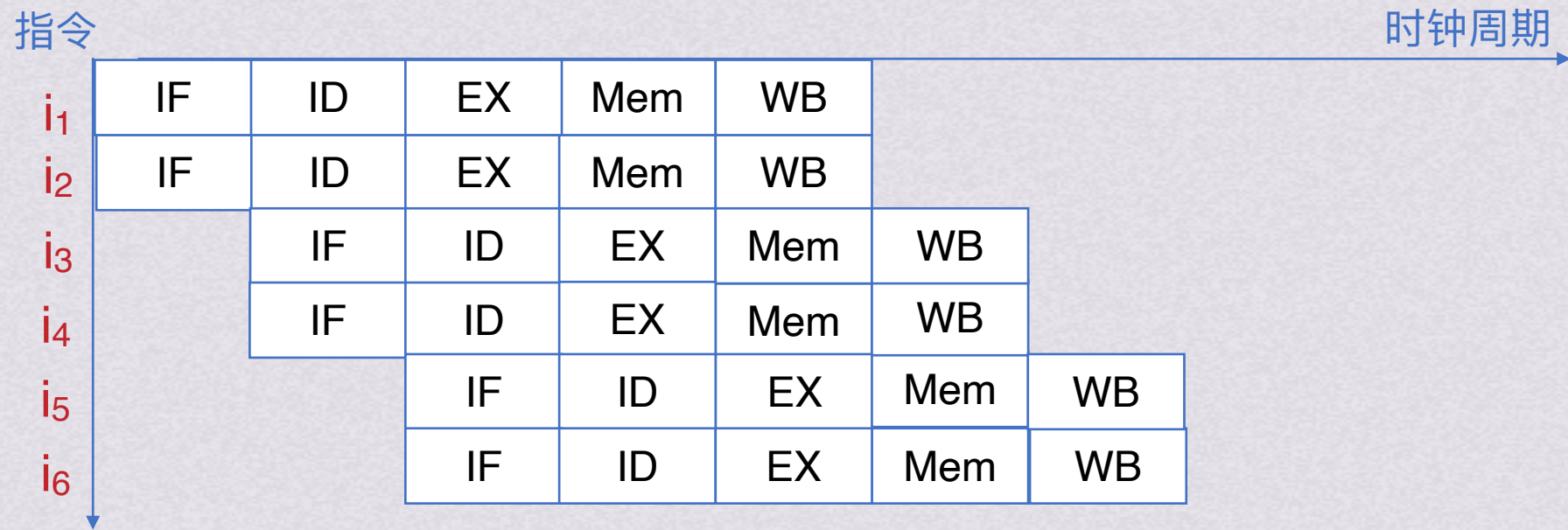
- 每个时钟周期内可同时并发多条独立指令，即以并行操作方式将两条或两条以上的指令编译并执行
- 要求处理机中配置多个功能部件和指令译码电路，以及多个寄存器端口和总线，以便能实现同时执行多个操作，此外还要编译程序决定哪几条相邻指令可以并行执行



# 流水线中的多发技术

## 超标量技术

- 每个时钟周期内可同时并发多条独立指令，即以并行操作方式将两条或两条以上的指令编译并执行
- 要求处理机中配置多个功能部件和指令译码电路，以及多个寄存器端口和总线，以便能实现同时执行多个操作，此外还要编译程序决定哪几条相邻指令可以并行执行



理想状态下，采用超标量流水线技术CPU的CPI可以 $< 1$

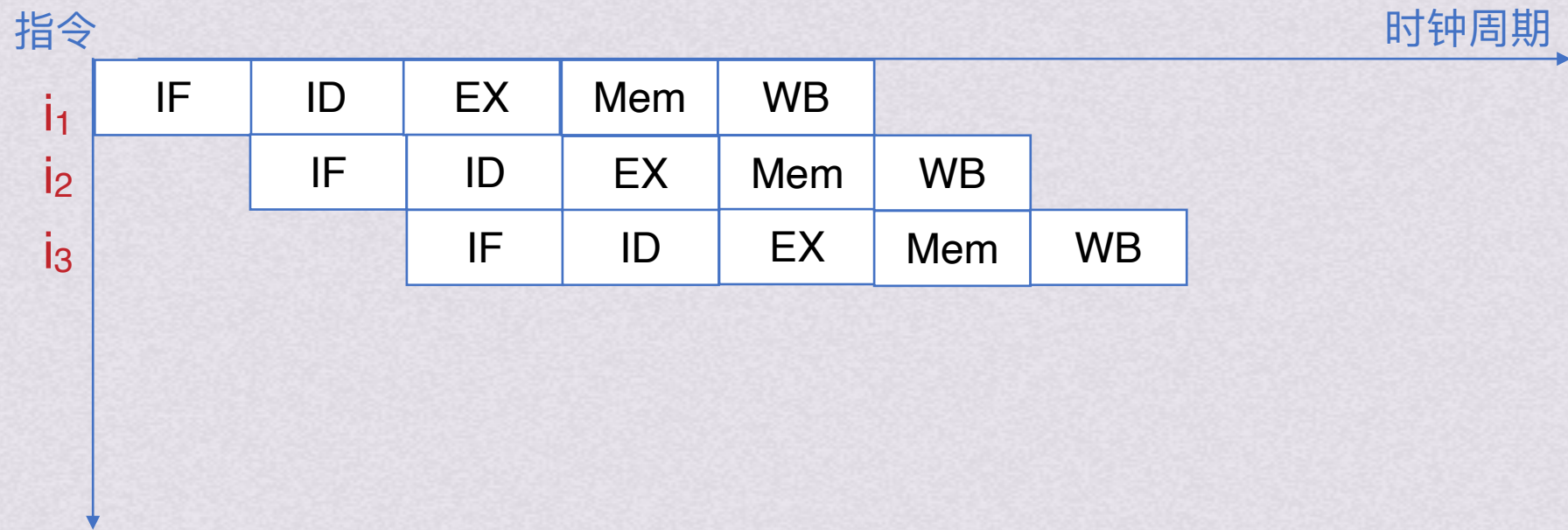


# 流水线中的多发技术

超标量技术

## 超流水线技术

- 将一些流水线寄存器插入到流水线段中，好比将流水线再分段，使流水线的处理器周期比普通流水线的处理器周期短
- 与超标量计算机一样，硬件不能调整指令的执行顺序，靠编译程序解决优化问题

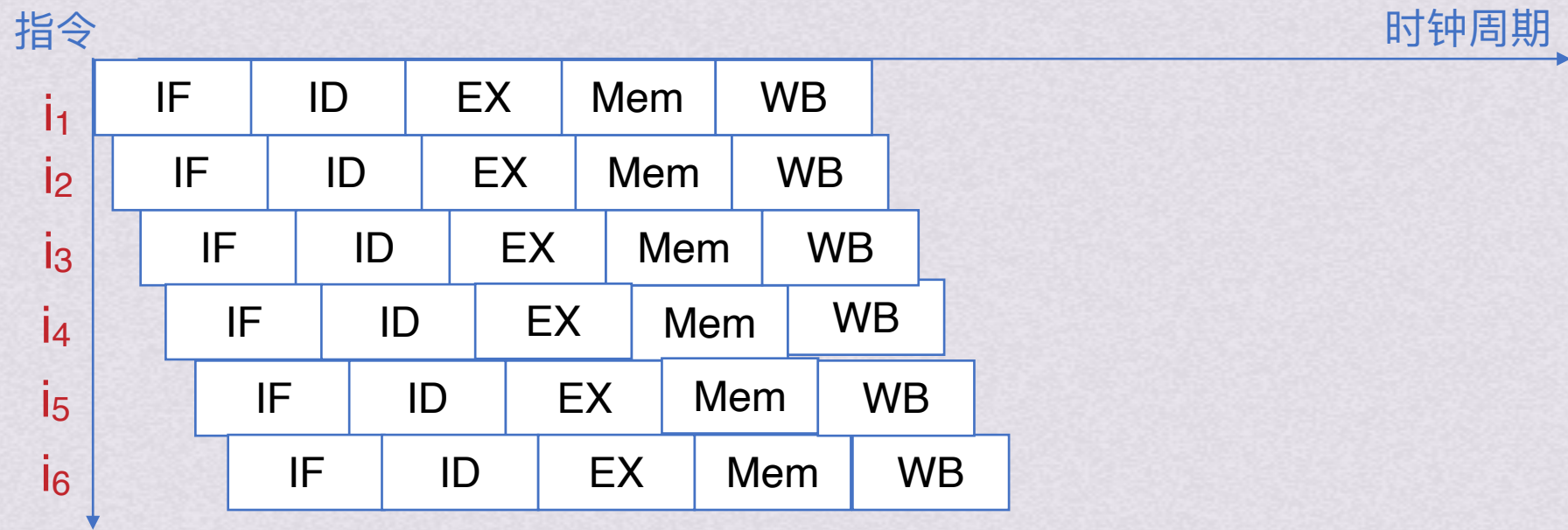




# 流水线中的多发技术 超标量技术

## 超流水线技术

- 将一些流水线寄存器插入到流水线段中，好比将流水线再分段，使流水线的处理器周期比普通流水线的处理器周期短
- 与超标量计算机一样，硬件不能调整指令的执行顺序，靠编译程序解决优化问题



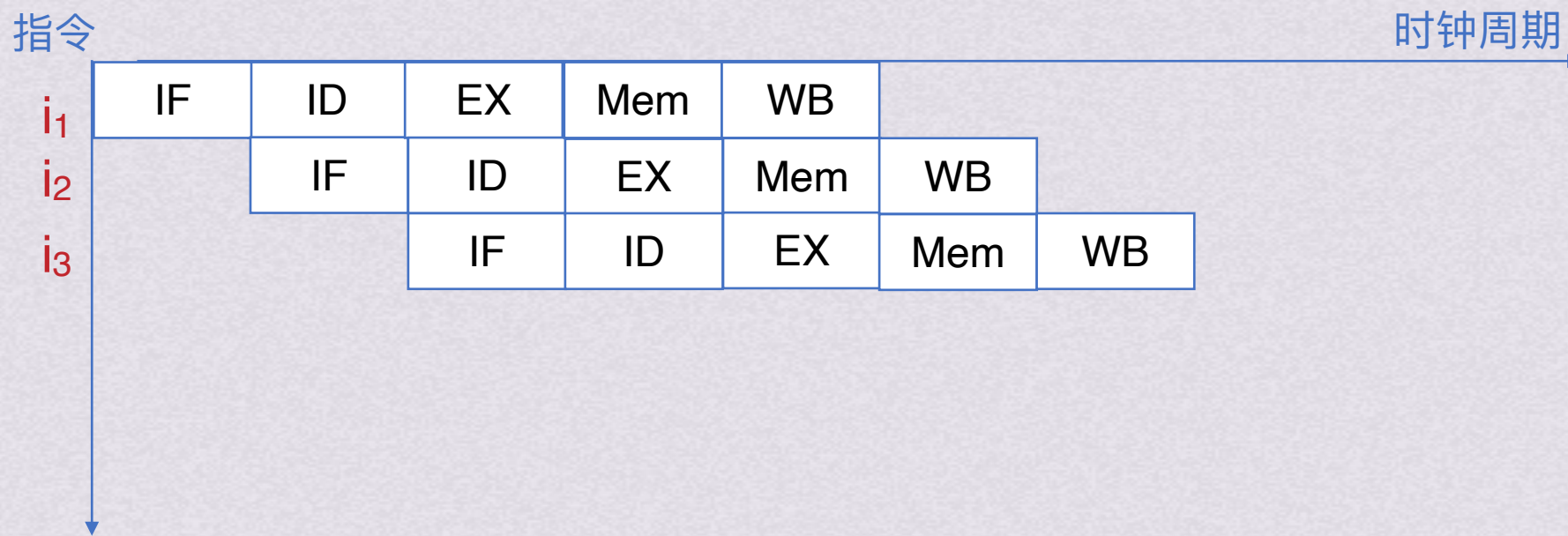


# 流水线中的多发技术

超标量技术 超流水线技术

## 超长指令字技术

- 由编译程序在编译时挖掘出指令间潜在的并行性，把多条能并行的指令组合成一条具有多个操作码字段的超长指令
- 具有更高的并行处理能力，但对优化编译器的要求更高，对cache容量要求更大



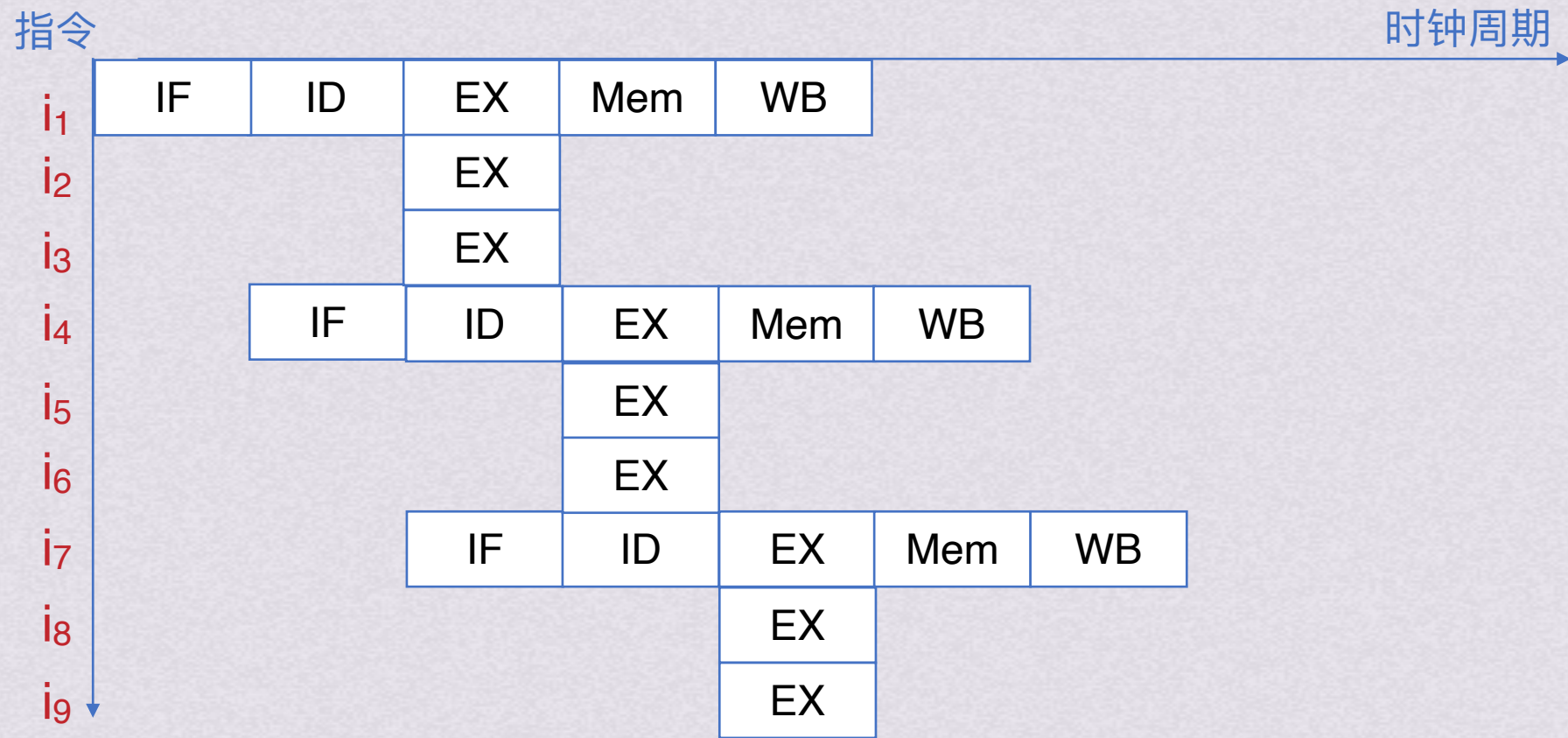


# 流水线中的多发技术

超标量技术 超流水线技术

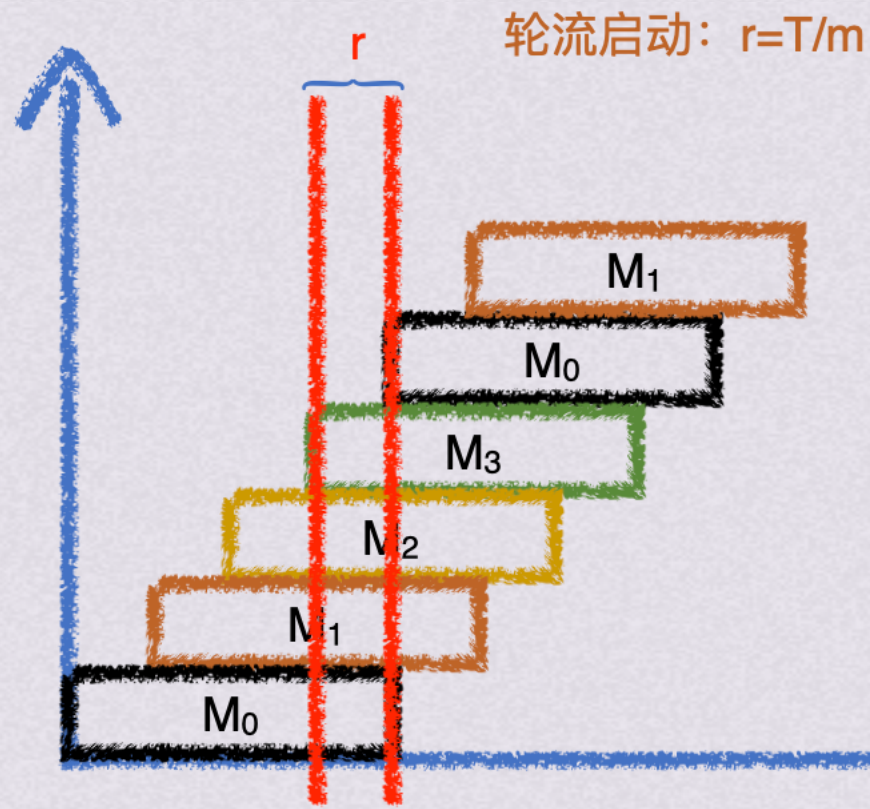
## 超长指令字技术

- 由编译程序在编译时挖掘出指令间潜在的并行性，把多条能并行的指令组合成一条具有多个操作码字段的超长指令
- 具有更高的并行处理能力，但对优化编译器的要求更高，对cache容量要求更大





以 $m=4$ 为例



由于轮流启动实现了存储体并行，  
因此进入流水线稳定状态后，完成一次  
数据传输所需的时间仅要 $T/m$ ，  
所以 $r=T/m$ ，时长被优化

时间